

Lecture 3: Linear Models for Regression

Tao LIN

SoE, Westlake University

June 15, 2025



- 1 Linear Regression
 - Gradient Descent (GD) variants for Linear Regression
 - Normal Equations and Least Squares
 - Probabilistic Interpretation of Linear Regression
- 2 Under-fitting, Polynomial Regression, and Over-fitting
 - Polynomial Features and Fitting Behavior
- 3 Regularization: Ridge Regression, and Lasso Regression
 - Ridge Regression
 - Lasso Regression
 - Another View of Regularization: Geometric Interpretation
 - Ridge Regression as MAP Estimator

Reading materials

- Chapter 1, Stanford CS 229 Lecture Notes,
https://cs229.stanford.edu/notes2022fall/main_notes.pdf
- Chapter 3.1 & 3.2, Bishop, Pattern Recognition and Machine Learning
- Read about over-fitting in the paper by Pedro Domingos (Sections 3 and 5 of “A few useful things to know about machine learning”)

Reference

- EPFL, CS-433 Machine Learning, https://github.com/epfml/ML_course

Session 1: How do we solve linear regression?

How can we solve linear regression—and when should we not form an inverse?

Three questions about linear regression

- 1 **How do we solve it?** MSE, GD/SGD, normal equations, and projection.
- 2 **What does the loss mean?** Noise, likelihood, polynomial features, and fitting behavior.
- 3 **How do we control complexity?** Ridge, lasso, and a probabilistic view.

Takeaway

Today's goal: choose a regression method and explain the choice.

Next lecture: generalization gap, model selection, and bias–variance.

Linear regression: predictions first, then residuals

Read the shapes: N examples, D features, one weight per feature.

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times D}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{w} \in \mathbb{R}^D.$$

Linear regression: predictions first, then residuals

Read the shapes: N examples, D features, one weight per feature.

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times D}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{w} \in \mathbb{R}^D.$$

Predict: $f_{\mathbf{w}}(\mathbf{X}) = \hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$, so $\hat{y}_n = \mathbf{x}_n^\top \mathbf{w}$.

Linear regression: predictions first, then residuals

Read the shapes: N examples, D features, one weight per feature.

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times D}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N, \quad \mathbf{w} \in \mathbb{R}^D.$$

Predict: $f_{\mathbf{w}}(\mathbf{X}) = \hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$, so $\hat{y}_n = \mathbf{x}_n^\top \mathbf{w}$.

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\mathbf{w}, \quad e_n = y_n - \mathbf{x}_n^\top \mathbf{w}.$$

Interpret: a positive residual means the prediction is too small.

From residuals to MSE, then to a gradient

Choose weights that make the residuals small. The factor $1/2$ simplifies derivatives but does not change the minimizer.

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N \underbrace{(y_n - \mathbf{x}_n^\top \mathbf{w})^2}_{e_n^2} = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2.$$

From residuals to MSE, then to a gradient

Choose weights that make the residuals small. The factor $1/2$ simplifies derivatives but does not change the minimizer.

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N \underbrace{(y_n - \mathbf{x}_n^\top \mathbf{w})^2}_{e_n^2} = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2.$$

$$\nabla_{\mathbf{w}} e_n = -\mathbf{x}_n, \quad \nabla_{\mathbf{w}} \frac{1}{2} e_n^2 = e_n \nabla_{\mathbf{w}} e_n = -\mathbf{x}_n e_n.$$

From residuals to MSE, then to a gradient

Choose weights that make the residuals small. The factor $1/2$ simplifies derivatives but does not change the minimizer.

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N \underbrace{(y_n - \mathbf{x}_n^\top \mathbf{w})^2}_{e_n^2} = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2.$$

$$\nabla_{\mathbf{w}} e_n = -\mathbf{x}_n, \quad \nabla_{\mathbf{w}} \frac{1}{2} e_n^2 = e_n \nabla_{\mathbf{w}} e_n = -\mathbf{x}_n e_n.$$

$$\nabla \mathcal{L}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N \mathbf{x}_n e_n = -\frac{1}{N} \mathbf{X}^\top \mathbf{e}.$$

From residuals to MSE, then to a gradient

Choose weights that make the residuals small. The factor $1/2$ simplifies derivatives but does not change the minimizer.

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N \underbrace{(y_n - \mathbf{x}_n^\top \mathbf{w})^2}_{e_n^2} = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2.$$

$$\nabla_{\mathbf{w}} e_n = -\mathbf{x}_n, \quad \nabla_{\mathbf{w}} \frac{1}{2} e_n^2 = e_n \nabla_{\mathbf{w}} e_n = -\mathbf{x}_n e_n.$$

$$\nabla \mathcal{L}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N \mathbf{x}_n e_n = -\frac{1}{N} \mathbf{X}^\top \mathbf{e}.$$

Takeaway

Batch GD follows the negative gradient: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \frac{\gamma}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^{(t)})$, with learning rate $\gamma > 0$.

A tiny regression step: follow every quantity

Question: for one example, will gradient descent move the prediction up or down?

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad y = 2$$

A tiny regression step: follow every quantity

Question: for one example, will gradient descent move the prediction up or down?

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad y = 2$$

$$\hat{y} = \mathbf{x}^\top \mathbf{w} = 1(1) + 2(-1) = -1.$$

A tiny regression step: follow every quantity

Question: for one example, will gradient descent move the prediction up or down?

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad y = 2$$

$$\hat{y} = \mathbf{x}^\top \mathbf{w} = 1(1) + 2(-1) = -1.$$

$$e = y - \hat{y} = 3 > 0.$$

The prediction is too small.

A tiny regression step: follow every quantity

Question: for one example, will gradient descent move the prediction up or down?

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad y = 2$$

$$\hat{y} = \mathbf{x}^\top \mathbf{w} = 1(1) + 2(-1) = -1.$$

$$\ell(\mathbf{w}) = \frac{1}{2}e^2 = \frac{9}{2}.$$

This is one-example loss, so there is no extra $1/N$.

$$e = y - \hat{y} = 3 > 0.$$

The prediction is too small.

A tiny regression step: follow every quantity

Question: for one example, will gradient descent move the prediction up or down?

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad y = 2$$

$$\hat{y} = \mathbf{x}^\top \mathbf{w} = 1(1) + 2(-1) = -1.$$

$$e = y - \hat{y} = 3 > 0.$$

The prediction is too small.

$$\ell(\mathbf{w}) = \frac{1}{2}e^2 = \frac{9}{2}.$$

This is one-example loss, so there is no extra $1/N$.

$$\nabla \ell(\mathbf{w}) = -\mathbf{x}e = \begin{bmatrix} -3 \\ -6 \end{bmatrix}.$$

A tiny regression step: follow every quantity

Question: for one example, will gradient descent move the prediction up or down?

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad y = 2$$

$$\hat{y} = \mathbf{x}^\top \mathbf{w} = 1(1) + 2(-1) = -1.$$

$$e = y - \hat{y} = 3 > 0.$$

The prediction is too small.

$$\ell(\mathbf{w}) = \frac{1}{2}e^2 = \frac{9}{2}.$$

This is one-example loss, so there is no extra $1/N$.

$$\nabla \ell(\mathbf{w}) = -\mathbf{x}e = \begin{bmatrix} -3 \\ -6 \end{bmatrix}.$$

Takeaway

$\mathbf{w}^+ = \mathbf{w} - \gamma \nabla \ell(\mathbf{w})$ increases both weights and moves $\hat{y}$ upward for a sufficiently small $\gamma > 0$.

Stochastic Gradient Descent (SGD): how much data per step?

Write $\mathcal{L} = \frac{1}{N} \sum_n \mathcal{L}_n$, where $\mathcal{L}_n(\mathbf{w}) = \frac{1}{2}(y_n - \mathbf{x}_n^\top \mathbf{w})^2$. Every method uses $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \gamma \mathbf{g}^{(t)}$.

Method	Examples per step	$\mathbf{g}^{(t)}$

Stochastic Gradient Descent (SGD): how much data per step?

Write $\mathcal{L} = \frac{1}{N} \sum_n \mathcal{L}_n$, where $\mathcal{L}_n(\mathbf{w}) = \frac{1}{2}(y_n - \mathbf{x}_n^\top \mathbf{w})^2$. Every method uses $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \gamma \mathbf{g}^{(t)}$.

Method	Examples per step	$\mathbf{g}^{(t)}$
Batch GD	N	$\nabla \mathcal{L}(\mathbf{w}^{(t)})$

Stochastic Gradient Descent (SGD): how much data per step?

Write $\mathcal{L} = \frac{1}{N} \sum_n \mathcal{L}_n$, where $\mathcal{L}_n(\mathbf{w}) = \frac{1}{2}(y_n - \mathbf{x}_n^\top \mathbf{w})^2$. Every method uses $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \gamma \mathbf{g}^{(t)}$.

Method	Examples per step	$\mathbf{g}^{(t)}$
Batch GD	N	$\nabla \mathcal{L}(\mathbf{w}^{(t)})$
SGD	1	$\nabla \mathcal{L}_n(\mathbf{w}^{(t)})$

Stochastic Gradient Descent (SGD): how much data per step?

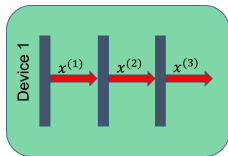
Write $\mathcal{L} = \frac{1}{N} \sum_n \mathcal{L}_n$, where $\mathcal{L}_n(\mathbf{w}) = \frac{1}{2}(y_n - \mathbf{x}_n^\top \mathbf{w})^2$. Every method uses $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \gamma \mathbf{g}^{(t)}$.

Method	Examples per step	$\mathbf{g}^{(t)}$
Batch GD	N	$\nabla \mathcal{L}(\mathbf{w}^{(t)})$
SGD	1	$\nabla \mathcal{L}_n(\mathbf{w}^{(t)})$
Mini-batch SGD	$ B $	$\frac{1}{ B } \sum_{n \in B} \nabla \mathcal{L}_n(\mathbf{w}^{(t)})$

Stochastic Gradient Descent (SGD): how much data per step?

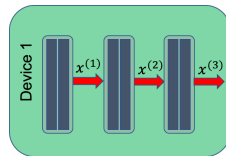
Write $\mathcal{L} = \frac{1}{N} \sum_n \mathcal{L}_n$, where $\mathcal{L}_n(\mathbf{w}) = \frac{1}{2}(y_n - \mathbf{x}_n^\top \mathbf{w})^2$. Every method uses $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \gamma \mathbf{g}^{(t)}$.

Method	Examples per step	$\mathbf{g}^{(t)}$
Batch GD	N	$\nabla \mathcal{L}(\mathbf{w}^{(t)})$
SGD	1	$\nabla \mathcal{L}_n(\mathbf{w}^{(t)})$
Mini-batch SGD	$ B $	$\frac{1}{ B } \sum_{n \in B} \nabla \mathcal{L}_n(\mathbf{w}^{(t)})$



SGD: cheap, noisy steps.

All gradients in a mini-batch use the *same* current weights.



Mini-batches: average several gradients.

Why can one example guide the whole objective?

If we sample n uniformly from $\{1, \dots, N\}$, is its gradient the full gradient on every step?

Why can one example guide the whole objective?

If we sample n uniformly from $\{1, \dots, N\}$, is its gradient the full gradient on every step?

Let the random index be $I \sim \text{Uniform}\{1, \dots, N\}$. One update uses the random vector $\nabla \mathcal{L}_I(\mathbf{w}^{(t)})$.

Why can one example guide the whole objective?

If we sample n uniformly from $\{1, \dots, N\}$, is its gradient the full gradient on every step?

Let the random index be $I \sim \text{Uniform}\{1, \dots, N\}$. One update uses the random vector $\nabla \mathcal{L}_I(\mathbf{w}^{(t)})$. **It is not the full gradient on each draw, but it is unbiased:**

$$\mathbb{E}_I[\nabla \mathcal{L}_I(\mathbf{w}^{(t)}) \mid \mathbf{w}^{(t)}] = \sum_{i=1}^N \frac{1}{N} \nabla \mathcal{L}_i(\mathbf{w}^{(t)}) = \nabla \mathcal{L}(\mathbf{w}^{(t)}).$$

A uniformly sampled mini-batch average is unbiased as well.

Why can one example guide the whole objective?

If we sample n uniformly from $\{1, \dots, N\}$, is its gradient the full gradient on every step?

Let the random index be $I \sim \text{Uniform}\{1, \dots, N\}$. One update uses the random vector $\nabla \mathcal{L}_I(\mathbf{w}^{(t)})$. **It is not the full gradient on each draw, but it is unbiased:**

$$\mathbb{E}_I[\nabla \mathcal{L}_I(\mathbf{w}^{(t)}) \mid \mathbf{w}^{(t)}] = \sum_{i=1}^N \frac{1}{N} \nabla \mathcal{L}_i(\mathbf{w}^{(t)}) = \nabla \mathcal{L}(\mathbf{w}^{(t)}).$$

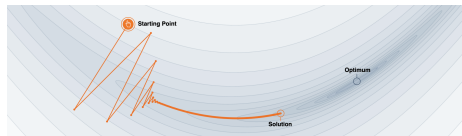
A uniformly sampled mini-batch average is unbiased as well.

Takeaway

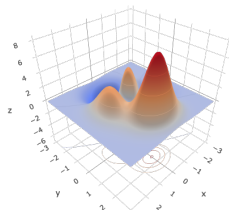
SGD trades a noisy direction for less work per update. Mini-batches average noise; the learning rate still controls stability.

Motivation: can we find the optimum without repeated steps?

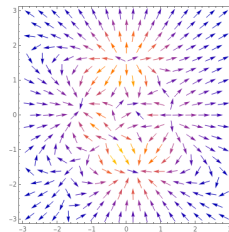
GD/SGD approach a minimizer iteratively. Least squares also has an analytical characterization.



Iterative updates



Generic function

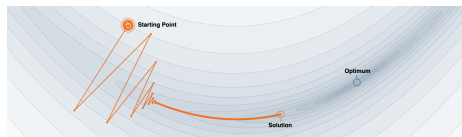


Its gradient field

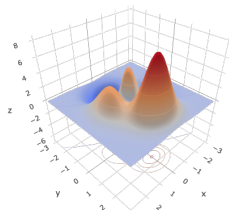
What equations must the best weights satisfy?

Motivation: can we find the optimum without repeated steps?

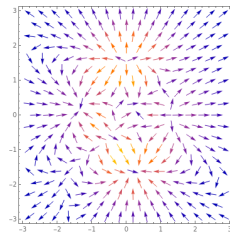
GD/SGD approach a minimizer iteratively. Least squares also has an analytical characterization.



Iterative updates



Generic function



Its gradient field

What equations must the best weights satisfy?

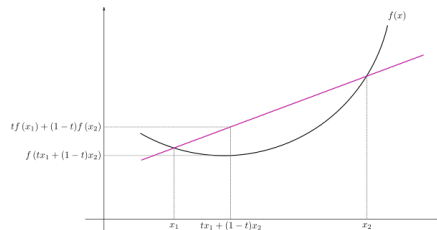
For the MSE, first justify that a zero gradient gives a global minimum; then solve $\nabla \mathcal{L}(\mathbf{w}^*) = 0$.

Steps to form normal equations: why convexity matters

Convexity: the line segment between two points on the graph never lies below the graph.

$$h(\lambda \mathbf{u} + (1 - \lambda) \mathbf{v}) \leq \lambda h(\mathbf{u}) + (1 - \lambda) h(\mathbf{v}),$$

for $0 \leq \lambda \leq 1$.

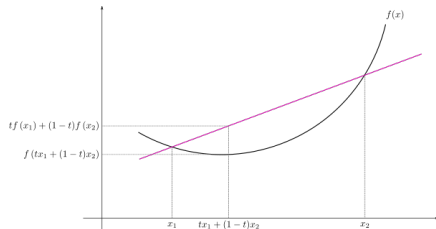


Steps to form normal equations: why convexity matters

Convexity: the line segment between two points on the graph never lies below the graph.

$$h(\lambda \mathbf{u} + (1 - \lambda) \mathbf{v}) \leq \lambda h(\mathbf{u}) + (1 - \lambda) h(\mathbf{v}),$$

for $0 \leq \lambda \leq 1$.



Takeaway

For a differentiable convex loss, a zero gradient is sufficient for a global minimum. Convexity alone does not guarantee uniqueness.

Next: verify this property for our MSE loss.

Derivation (step 1): the MSE is convex in the weights

Differentiate the gradient once more:

$$\nabla \mathcal{L}(\mathbf{w}) = \frac{1}{N}(\mathbf{X}^\top \mathbf{X} \mathbf{w} - \mathbf{X}^\top \mathbf{y}), \quad \nabla^2 \mathcal{L}(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top \mathbf{X}.$$

Derivation (step 1): the MSE is convex in the weights

Differentiate the gradient once more:

$$\nabla \mathcal{L}(\mathbf{w}) = \frac{1}{N}(\mathbf{X}^\top \mathbf{X} \mathbf{w} - \mathbf{X}^\top \mathbf{y}), \quad \nabla^2 \mathcal{L}(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top \mathbf{X}.$$

Test any direction $\mathbf{v} \in \mathbb{R}^D$:

$$\mathbf{v}^\top \nabla^2 \mathcal{L}(\mathbf{w}) \mathbf{v} = \frac{1}{N} (\mathbf{X} \mathbf{v})^\top (\mathbf{X} \mathbf{v}) = \frac{1}{N} \|\mathbf{X} \mathbf{v}\|_2^2 \geq 0.$$

The Hessian is positive semidefinite, so the MSE is convex.

Derivation (step 1): the MSE is convex in the weights

Differentiate the gradient once more:

$$\nabla \mathcal{L}(\mathbf{w}) = \frac{1}{N}(\mathbf{X}^\top \mathbf{X} \mathbf{w} - \mathbf{X}^\top \mathbf{y}), \quad \nabla^2 \mathcal{L}(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top \mathbf{X}.$$

Test any direction $\mathbf{v} \in \mathbb{R}^D$:

$$\mathbf{v}^\top \nabla^2 \mathcal{L}(\mathbf{w}) \mathbf{v} = \frac{1}{N} (\mathbf{X} \mathbf{v})^\top (\mathbf{X} \mathbf{v}) = \frac{1}{N} \|\mathbf{X} \mathbf{v}\|_2^2 \geq 0.$$

The Hessian is positive semidefinite, so the MSE is convex.

If $\mathbf{X}$ has full column rank, this quantity is positive for $\mathbf{v} \neq \mathbf{0}$: the loss is strictly convex and the minimizer is unique.

Derivation (step 2): set the gradient to zero

Keep the residual convention $\mathbf{e} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

$$\nabla \mathcal{L}(\mathbf{w}^*) = -\frac{1}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^*) = \mathbf{0}$$

Derivation (step 2): set the gradient to zero

Keep the residual convention $\mathbf{e} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

$$\begin{aligned}\nabla \mathcal{L}(\mathbf{w}^*) &= -\frac{1}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^*) = \mathbf{0} \\ \iff \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^*) &= \mathbf{0}\end{aligned}$$

Derivation (step 2): set the gradient to zero

Keep the residual convention $\mathbf{e} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

$$\begin{aligned}\nabla \mathcal{L}(\mathbf{w}^*) &= -\frac{1}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^*) = \mathbf{0} \\ \iff \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^*) &= \mathbf{0} \\ \iff \underbrace{\mathbf{X}^\top \mathbf{X}}_{D \times D} \mathbf{w}^* &= \underbrace{\mathbf{X}^\top \mathbf{y}}_{D \times 1}.\end{aligned}$$

Derivation (step 2): set the gradient to zero

Keep the residual convention $\mathbf{e} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

$$\begin{aligned}\nabla \mathcal{L}(\mathbf{w}^*) &= -\frac{1}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^*) = \mathbf{0} \\ \iff \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^*) &= \mathbf{0} \\ \iff \underbrace{\mathbf{X}^\top \mathbf{X}}_{D \times D} \mathbf{w}^* &= \underbrace{\mathbf{X}^\top \mathbf{y}}_{D \times 1}.\end{aligned}$$

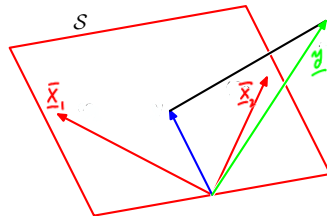
These are the [normal equations](#). Convexity makes every solution an MSE minimizer, even when the coefficients are not unique.

Geometric interpretation of Normal Equations

Which predictions are possible?

$$\mathcal{S} = \text{col}(\mathbf{X}) = \{\mathbf{X}\mathbf{w} : \mathbf{w} \in \mathbb{R}^D\} \subseteq \mathbb{R}^N.$$

This is the span of the *columns* of $\mathbf{X}$; each column is a vector across all N examples.



Bishop: projection in data space

Geometric interpretation of Normal Equations

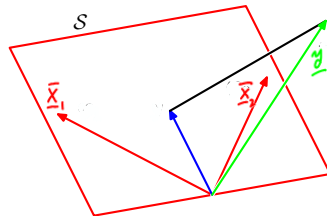
Which predictions are possible?

$$\mathcal{S} = \text{col}(\mathbf{X}) = \{\mathbf{X}\mathbf{w} : \mathbf{w} \in \mathbb{R}^D\} \subseteq \mathbb{R}^N.$$

This is the span of the *columns* of $\mathbf{X}$; each column is a vector across all N examples.

$$\mathbf{X}^\top \mathbf{e}^* = \mathbf{0} \implies \mathbf{e}^* \perp \text{col}(\mathbf{X}).$$

The residual is orthogonal to every feature column.



Bishop: projection in data space

Geometric interpretation of Normal Equations

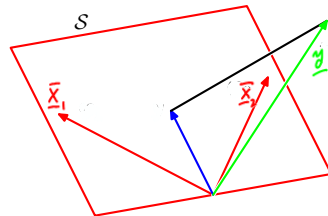
Which predictions are possible?

$$\mathcal{S} = \text{col}(\mathbf{X}) = \{\mathbf{X}\mathbf{w} : \mathbf{w} \in \mathbb{R}^D\} \subseteq \mathbb{R}^N.$$

This is the span of the *columns* of $\mathbf{X}$; each column is a vector across all N examples.

$$\mathbf{X}^\top \mathbf{e}^* = \mathbf{0} \implies \mathbf{e}^* \perp \text{col}(\mathbf{X}).$$

The residual is orthogonal to every feature column.



Bishop: projection in data space

Takeaway

$\hat{\mathbf{y}}^* = \mathbf{X}\mathbf{w}^*$ is the orthogonal projection of $\mathbf{y}$ onto $\text{col}(\mathbf{X})$: the closest attainable prediction.

Why does an orthogonal residual minimize squared error?

Starting at $\hat{\mathbf{y}}^*$, change the weights by $\Delta \mathbf{w}$. Can the new prediction $\hat{\mathbf{y}}^* + \mathbf{X}\Delta \mathbf{w}$ be closer to $\mathbf{y}$?

Why does an orthogonal residual minimize squared error?

Starting at $\hat{\mathbf{y}}^*$, change the weights by $\Delta \mathbf{w}$. Can the new prediction $\hat{\mathbf{y}}^* + \mathbf{X}\Delta \mathbf{w}$ be closer to $\mathbf{y}$?

$$\begin{aligned}\|\mathbf{y} - (\hat{\mathbf{y}}^* + \mathbf{X}\Delta \mathbf{w})\|_2^2 &= \|\mathbf{e}^* - \mathbf{X}\Delta \mathbf{w}\|_2^2 \\ &= \|\mathbf{e}^*\|_2^2 - 2(\mathbf{e}^*)^\top \mathbf{X}\Delta \mathbf{w} + \|\mathbf{X}\Delta \mathbf{w}\|_2^2.\end{aligned}$$

Why does an orthogonal residual minimize squared error?

Starting at $\hat{\mathbf{y}}^*$, change the weights by $\Delta \mathbf{w}$. Can the new prediction $\hat{\mathbf{y}}^* + \mathbf{X}\Delta \mathbf{w}$ be closer to $\mathbf{y}$?

$$\begin{aligned}\|\mathbf{y} - (\hat{\mathbf{y}}^* + \mathbf{X}\Delta \mathbf{w})\|_2^2 &= \|\mathbf{e}^* - \mathbf{X}\Delta \mathbf{w}\|_2^2 \\ &= \|\mathbf{e}^*\|_2^2 - 2(\mathbf{e}^*)^\top \mathbf{X}\Delta \mathbf{w} + \|\mathbf{X}\Delta \mathbf{w}\|_2^2.\end{aligned}$$

Normal equations give $\mathbf{X}^\top \mathbf{e}^* = \mathbf{0}$, hence $(\mathbf{e}^*)^\top \mathbf{X}\Delta \mathbf{w} = 0$.

Why does an orthogonal residual minimize squared error?

Starting at $\hat{\mathbf{y}}^*$, change the weights by $\Delta \mathbf{w}$. Can the new prediction $\hat{\mathbf{y}}^* + \mathbf{X}\Delta \mathbf{w}$ be closer to $\mathbf{y}$?

$$\begin{aligned}\|\mathbf{y} - (\hat{\mathbf{y}}^* + \mathbf{X}\Delta \mathbf{w})\|_2^2 &= \|\mathbf{e}^* - \mathbf{X}\Delta \mathbf{w}\|_2^2 \\ &= \|\mathbf{e}^*\|_2^2 - 2(\mathbf{e}^*)^\top \mathbf{X}\Delta \mathbf{w} + \|\mathbf{X}\Delta \mathbf{w}\|_2^2.\end{aligned}$$

Normal equations give $\mathbf{X}^\top \mathbf{e}^* = \mathbf{0}$, hence $(\mathbf{e}^*)^\top \mathbf{X}\Delta \mathbf{w} = 0$.

$$\|\mathbf{y} - (\hat{\mathbf{y}}^* + \mathbf{X}\Delta \mathbf{w})\|_2^2 = \|\mathbf{e}^*\|_2^2 + \|\mathbf{X}\Delta \mathbf{w}\|_2^2 \geq \|\mathbf{e}^*\|_2^2.$$

Answer: no. A different fitted vector increases the squared error. A weight change with $\mathbf{X}\Delta \mathbf{w} = \mathbf{0}$ leaves the fit unchanged.

Least Squares: a closed form when columns are independent

Assumption: $\text{rank}(\mathbf{X}) = D$ (full column rank). Then the Gram matrix $\mathbf{X}^\top \mathbf{X}$ is invertible.

$$\mathbf{X}^\top \mathbf{X} \mathbf{w}^* = \mathbf{X}^\top \mathbf{y}$$

Least Squares: a closed form when columns are independent

Assumption: $\text{rank}(\mathbf{X}) = D$ (full column rank). Then the Gram matrix $\mathbf{X}^\top \mathbf{X}$ is invertible.

$$\mathbf{X}^\top \mathbf{X} \mathbf{w}^* = \mathbf{X}^\top \mathbf{y}$$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

This is the unique ordinary least-squares (OLS) coefficient vector. For a new example, predict $\hat{y}_m = \mathbf{x}_m^\top \mathbf{w}^*$.

Least Squares: a closed form when columns are independent

Assumption: $\text{rank}(\mathbf{X}) = D$ (full column rank). Then the Gram matrix $\mathbf{X}^\top \mathbf{X}$ is invertible.

$$\mathbf{X}^\top \mathbf{X} \mathbf{w}^* = \mathbf{X}^\top \mathbf{y}$$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

This is the unique ordinary least-squares (OLS) coefficient vector. For a new example, predict $\hat{y}_m = \mathbf{x}_m^\top \mathbf{w}^*$.

Takeaway

The inverse is a mathematical expression for the optimizer. It is not an instruction to form an inverse in numerical code.

Rank deficiency: different weights can give the same fit

When can full column rank fail? If $D > N$, or if feature columns are exactly linearly dependent.

Rank deficiency: different weights can give the same fit

When can full column rank fail? If $D > N$, or if feature columns are exactly linearly dependent. For example, if columns 1 and 2 are identical,

$$\mathbf{X}(\mathbf{w} + a(1, -1, 0, \dots, 0)^\top) = \mathbf{X}\mathbf{w} \quad \text{for every } a \in \mathbb{R}.$$

More generally, $\mathbf{X}\mathbf{v} = \mathbf{0}$ implies $\mathbf{X}(\mathbf{w}^* + \mathbf{v}) = \mathbf{X}\mathbf{w}^*$.

Rank deficiency: different weights can give the same fit

When can full column rank fail? If $D > N$, or if feature columns are exactly linearly dependent. For example, if columns 1 and 2 are identical,

$$\mathbf{X}(\mathbf{w} + a(1, -1, 0, \dots, 0)^\top) = \mathbf{X}\mathbf{w} \quad \text{for every } a \in \mathbb{R}.$$

More generally, $\mathbf{X}\mathbf{v} = \mathbf{0}$ implies $\mathbf{X}(\mathbf{w}^* + \mathbf{v}) = \mathbf{X}\mathbf{w}^*$.

Takeaway

Rank deficiency can yield multiple coefficient vectors with the same fitted values. The least-squares fitted vector is still the unique projection; the inverse formula is unavailable.

III-conditioning: how should we compute least squares?

The spectral condition number κ_2 measures numerical sensitivity: larger κ_2 means greater sensitivity, as with nearly dependent columns.

$$\kappa_2(\mathbf{X}^\top \mathbf{X}) = \kappa_2(\mathbf{X})^2 \quad \text{when } \mathbf{X} \text{ has full column rank.}$$

Forming $\mathbf{X}^\top \mathbf{X}$ squares the spectral condition number.

III-conditioning: how should we compute least squares?

The spectral condition number κ_2 measures numerical sensitivity: larger κ_2 means greater sensitivity, as with nearly dependent columns.

$$\kappa_2(\mathbf{X}^\top \mathbf{X}) = \kappa_2(\mathbf{X})^2 \quad \text{when } \mathbf{X} \text{ has full column rank.}$$

Forming $\mathbf{X}^\top \mathbf{X}$ squares the spectral condition number.

Use QR for a full-rank least-squares problem; use SVD or a rank-revealing QR method when rank deficiency is possible. `lstsq` returns a minimum-norm solution if minimizers are non-unique.

Ill-conditioning: how should we compute least squares?

The spectral condition number κ_2 measures numerical sensitivity: larger κ_2 means greater sensitivity, as with nearly dependent columns.

$$\kappa_2(\mathbf{X}^\top \mathbf{X}) = \kappa_2(\mathbf{X})^2 \quad \text{when } \mathbf{X} \text{ has full column rank.}$$

Forming $\mathbf{X}^\top \mathbf{X}$ squares the spectral condition number.

Use QR for a full-rank least-squares problem; use SVD or a rank-revealing QR method when rank deficiency is possible. `lstsq` returns a minimum-norm solution if minimizers are non-unique.

Takeaway

The inverse formula explains the solution; numerical code should solve the least-squares problem with QR/SVD, for example `np.linalg.lstsq(X, y, rcond=None)`.

Do not explicitly form the inverse. A stable solver avoids unnecessary numerical error; it does not remove sensitivity in the data.

From mathematical optimum to computational choice

Which fact about the problem should determine how we compute the fit?

From mathematical optimum to computational choice

Which fact about the problem should determine how we compute the fit?

Moderate dense problem $\rightarrow$ QR-backed `lstsq`: avoid squaring the condition number.

From mathematical optimum to computational choice

Which fact about the problem should determine how we compute the fit?

Moderate dense problem $\rightarrow$ QR-backed `lstsq`: avoid squaring the condition number.

Rank uncertain or ill-conditioned $\rightarrow$ SVD or rank-revealing QR: expose weak or null directions.

From mathematical optimum to computational choice

Which fact about the problem should determine how we compute the fit?

Moderate dense problem $\rightarrow$ QR-backed `lstsq`: avoid squaring the condition number.

Rank uncertain or ill-conditioned $\rightarrow$ SVD or rank-revealing QR: expose weak or null directions.

Very large or streaming data $\rightarrow$ mini-batch SGD: use cheap updates, then check convergence.

From mathematical optimum to computational choice

Which fact about the problem should determine how we compute the fit?

Moderate dense problem $\rightarrow$ QR-backed `lstsq`: avoid squaring the condition number.

Rank uncertain or ill-conditioned $\rightarrow$ SVD or rank-revealing QR: expose weak or null directions.

Very large or streaming data $\rightarrow$ mini-batch SGD: use cheap updates, then check convergence.

Takeaway

Normal equations characterize the optimum; a factorization or iterative method computes it. Do not explicitly form $(\mathbf{X}^\top \mathbf{X})^{-1}$.

Session 1: explain the method and the residual

A design has two identical feature columns. A classmate proposes $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. Explain (1) how to compute the fit, (2) whether weights are unique, and (3) where the residual points.

Session 1: explain the method and the residual

A design has two identical feature columns. A classmate proposes $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. Explain (1) how to compute the fit, (2) whether weights are unique, and (3) where the residual points.

First clue: identical columns make $\mathbf{X}$ rank-deficient, so $\mathbf{X}^\top \mathbf{X}$ is singular.

Session 1: explain the method and the residual

A design has two identical feature columns. A classmate proposes $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. Explain (1) how to compute the fit, (2) whether weights are unique, and (3) where the residual points.

First clue: identical columns make $\mathbf{X}$ rank-deficient, so $\mathbf{X}^\top \mathbf{X}$ is singular. **Compute:** the inverse does not exist. Use `np.linalg.lstsq` (or SVD/rank-revealing QR).

Session 1: explain the method and the residual

A design has two identical feature columns. A classmate proposes $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. Explain (1) how to compute the fit, (2) whether weights are unique, and (3) where the residual points.

First clue: identical columns make $\mathbf{X}$ rank-deficient, so $\mathbf{X}^\top \mathbf{X}$ is singular. **Compute:** the inverse does not exist. Use `np.linalg.lstsq` (or SVD/rank-revealing QR). Weights need not be unique, but fitted values are unique and $\mathbf{e}^* \perp \text{col}(\mathbf{X})$.

Session 1: explain the method and the residual

A design has two identical feature columns. A classmate proposes $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. Explain (1) how to compute the fit, (2) whether weights are unique, and (3) where the residual points.

First clue: identical columns make $\mathbf{X}$ rank-deficient, so $\mathbf{X}^\top \mathbf{X}$ is singular. **Compute:** the inverse does not exist. Use `np.linalg.lstsq` (or SVD/rank-revealing QR). Weights need not be unique, but fitted values are unique and $\mathbf{e}^* \perp \text{col}(\mathbf{X})$.

Method choice: with very large data, mini-batch SGD may offer cheaper updates; a converged least-squares optimum satisfies the same normal equations. Next: why choose squared error in the first place?

Optional: arbitrary sampling

Session 2: What does the loss mean?

We minimized squared error. What assumption about the observations makes that choice reasonable?

Gaussian noise: a model for repeated observations

Condition on the observed inputs $\mathbf{X}$. Assume

$$y_n = \mathbf{x}_n^\top \mathbf{w} + \epsilon_n, \quad \epsilon_n \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

Gaussian noise: a model for repeated observations

Condition on the observed inputs $\mathbf{X}$. Assume

$$y_n = \mathbf{x}_n^\top \mathbf{w} + \epsilon_n, \quad \epsilon_n \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

The noise is independent of the inputs; $\sigma^2 > 0$ is shared by all examples. The unknown weights determine the conditional mean:

$$\mathbb{E}[y_n \mid \mathbf{x}_n, \mathbf{w}] = \mathbf{x}_n^\top \mathbf{w}, \quad \text{Var}(y_n \mid \mathbf{x}_n, \mathbf{w}) = \sigma^2.$$

Gaussian noise: a model for repeated observations

Condition on the observed inputs $\mathbf{X}$. Assume

$$y_n = \mathbf{x}_n^\top \mathbf{w} + \epsilon_n, \quad \epsilon_n \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

The noise is independent of the inputs; $\sigma^2 > 0$ is shared by all examples. The unknown weights determine the conditional mean:

$$\mathbb{E}[y_n \mid \mathbf{x}_n, \mathbf{w}] = \mathbf{x}_n^\top \mathbf{w}, \quad \text{Var}(y_n \mid \mathbf{x}_n, \mathbf{w}) = \sigma^2.$$

Takeaway

A prediction is the center of a distribution of possible observations. The noise model describes how observations vary around it.

One observation: how plausible is its residual?

Write $e_n = y_n - \mathbf{x}_n^\top \mathbf{w}$. The Gaussian density is

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{e_n^2}{2\sigma^2}\right).$$

With the same σ , compare residuals $e_n = \sigma$ and $e_n = 2\sigma$. Which observation has the smaller density?

One observation: how plausible is its residual?

Write $e_n = y_n - \mathbf{x}_n^\top \mathbf{w}$. The Gaussian density is

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{e_n^2}{2\sigma^2}\right).$$

With the same σ , compare residuals $e_n = \sigma$ and $e_n = 2\sigma$. Which observation has the smaller density?

The exponents are $-1/2$ and -2 : the larger residual is less plausible. Squaring the residual treats positive and negative deviations equally.

One observation: how plausible is its residual?

Write $e_n = y_n - \mathbf{x}_n^\top \mathbf{w}$. The Gaussian density is

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{e_n^2}{2\sigma^2}\right).$$

With the same σ , compare residuals $e_n = \sigma$ and $e_n = 2\sigma$. Which observation has the smaller density?

The exponents are $-1/2$ and -2 : the larger residual is less plausible. Squaring the residual treats positive and negative deviations equally.

We compare *densities*; a continuous variable has probability zero of taking any one exact value. Next, combine all N observations.

Probability model versus likelihood: what is varying?

Before observing y : probability model

$$p(y \mid \mathbf{x}, \mathbf{w})$$

Fix $(\mathbf{x}, \mathbf{w})$ and ask which responses y the model could generate.

Probability model versus likelihood: what is varying?

Before observing y : probability model

$$p(y \mid \mathbf{x}, \mathbf{w})$$

Fix $(\mathbf{x}, \mathbf{w})$ and ask which responses y the model could generate.

After observing $(\mathbf{x}, y)$: likelihood

$$L(\mathbf{w}; \mathbf{x}, y) = p(y \mid \mathbf{x}, \mathbf{w})$$

Fix the data and compare candidate weights $\mathbf{w}$.

Probability model versus likelihood: what is varying?

Before observing y : probability model

$$p(y \mid \mathbf{x}, \mathbf{w})$$

Fix $(\mathbf{x}, \mathbf{w})$ and ask which responses y the model could generate.

After observing $(\mathbf{x}, y)$: likelihood

$$L(\mathbf{w}; \mathbf{x}, y) = p(y \mid \mathbf{x}, \mathbf{w})$$

Fix the data and compare candidate weights $\mathbf{w}$.

Let $y = 3$ and keep σ fixed. Candidate A predicts 2; candidate B predicts 0. Which candidate has larger likelihood?

Probability model versus likelihood: what is varying?

Before observing y : probability model

$$p(y \mid \mathbf{x}, \mathbf{w})$$

Fix $(\mathbf{x}, \mathbf{w})$ and ask which responses y the model could generate.

After observing $(\mathbf{x}, y)$: likelihood

$$L(\mathbf{w}; \mathbf{x}, y) = p(y \mid \mathbf{x}, \mathbf{w})$$

Fix the data and compare candidate weights $\mathbf{w}$.

Let $y = 3$ and keep σ fixed. Candidate A predicts 2; candidate B predicts 0. Which candidate has larger likelihood?

$|3 - 2| < |3 - 0|$, so $L(\mathbf{w}_A) > L(\mathbf{w}_B)$ under Gaussian noise. Likelihood *ranks* parameters; without a prior it is not a normalized probability distribution over parameters.

From one observation to the likelihood of the sample

With $\mathbf{X}$ fixed, conditional independence gives

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(y_n \mid \mathbf{x}_n^\top \mathbf{w}, \sigma^2).$$

From one observation to the likelihood of the sample

With $\mathbf{X}$ fixed, conditional independence gives

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(y_n \mid \mathbf{x}_n^\top \mathbf{w}, \sigma^2).$$

Substituting the density combines the exponents:

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = (2\pi\sigma^2)^{-N/2} \exp \left(-\frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \right).$$

From one observation to the likelihood of the sample

With $\mathbf{X}$ fixed, conditional independence gives

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(y_n \mid \mathbf{x}_n^\top \mathbf{w}, \sigma^2).$$

Substituting the density combines the exponents:

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = (2\pi\sigma^2)^{-N/2} \exp \left(-\frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \right).$$

The observed $(\mathbf{X}, \mathbf{y})$ and σ^2 are now fixed; changing $\mathbf{w}$ changes every residual and therefore this joint density.

From one observation to the likelihood of the sample

With $\mathbf{X}$ fixed, conditional independence gives

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(y_n \mid \mathbf{x}_n^\top \mathbf{w}, \sigma^2).$$

Substituting the density combines the exponents:

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = (2\pi\sigma^2)^{-N/2} \exp \left(-\frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \right).$$

The observed $(\mathbf{X}, \mathbf{y})$ and σ^2 are now fixed; changing $\mathbf{w}$ changes every residual and therefore this joint density.

Takeaway

Fix the observed data and vary $\mathbf{w}$. Maximum likelihood chooses weights that give these observations the largest joint density under the assumed model.

Take a logarithm: products become sums

The logarithm is increasing, so maximizing likelihood also maximizes log-likelihood. Negating it gives a minimization problem.

$$-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = -\sum_{n=1}^N \log p(y_n \mid \mathbf{x}_n, \mathbf{w})$$

Take a logarithm: products become sums

The logarithm is increasing, so maximizing likelihood also maximizes log-likelihood. Negating it gives a minimization problem.

$$\begin{aligned} -\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) &= -\sum_{n=1}^N \log p(y_n \mid \mathbf{x}_n, \mathbf{w}) \\ &= \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 + \underbrace{\frac{N}{2} \log(2\pi\sigma^2)}_C. \end{aligned}$$

Take a logarithm: products become sums

The logarithm is increasing, so maximizing likelihood also maximizes log-likelihood. Negating it gives a minimization problem.

$$\begin{aligned} -\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) &= -\sum_{n=1}^N \log p(y_n \mid \mathbf{x}_n, \mathbf{w}) \\ &= \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 + \underbrace{\frac{N}{2} \log(2\pi\sigma^2)}_C. \end{aligned}$$

For fixed $\sigma^2 > 0$, C and the positive scale $1/(2\sigma^2)$ do not depend on $\mathbf{w}$.

Take a logarithm: products become sums

The logarithm is increasing, so maximizing likelihood also maximizes log-likelihood. Negating it gives a minimization problem.

$$\begin{aligned} -\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) &= -\sum_{n=1}^N \log p(y_n \mid \mathbf{x}_n, \mathbf{w}) \\ &= \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 + \underbrace{\frac{N}{2} \log(2\pi\sigma^2)}_C. \end{aligned}$$

For fixed $\sigma^2 > 0$, C and the positive scale $1/(2\sigma^2)$ do not depend on $\mathbf{w}$.

Takeaway

The negative log-likelihood is scaled squared error plus C .

Interpretation: Gaussian MLE recovers least squares

Recall our half-MSE $\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$.

$$-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \frac{N}{\sigma^2} \mathcal{L}(\mathbf{w}) + C.$$

Does adding C or multiplying by $N/\sigma^2 > 0$ change which weights minimize the objective?

Interpretation: Gaussian MLE recovers least squares

Recall our half-MSE $\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$.

$$-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \frac{N}{\sigma^2} \mathcal{L}(\mathbf{w}) + C.$$

Does adding C or multiplying by $N/\sigma^2 > 0$ change which weights minimize the objective?

No. Adding a constant and multiplying by a positive number preserve every comparison between candidate weights.

Interpretation: Gaussian MLE recovers least squares

Recall our half-MSE $\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$.

$$-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \frac{N}{\sigma^2} \mathcal{L}(\mathbf{w}) + C.$$

Does adding C or multiplying by $N/\sigma^2 > 0$ change which weights minimize the objective?

No. Adding a constant and multiplying by a positive number preserve every comparison between candidate weights.

$$\arg \max_{\mathbf{w}} p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \arg \min_{\mathbf{w}} \mathcal{L}(\mathbf{w}).$$

Interpretation: Gaussian MLE recovers least squares

Recall our half-MSE $\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$.

$$-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \frac{N}{\sigma^2} \mathcal{L}(\mathbf{w}) + C.$$

Does adding C or multiplying by $N/\sigma^2 > 0$ change which weights minimize the objective?

No. Adding a constant and multiplying by a positive number preserve every comparison between candidate weights.

$$\arg \max_{\mathbf{w}} p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \arg \min_{\mathbf{w}} \mathcal{L}(\mathbf{w}).$$

Squared error encodes a conditional Gaussian noise assumption. We can still use least squares without assuming that the model is true.

Transfer check: change the noise, derive the loss

Assume independent Laplace noise with fixed scale $b > 0$:

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \frac{1}{2b} \exp \left(-\frac{|y_n - \mathbf{x}_n^\top \mathbf{w}|}{b} \right).$$

Take the negative logarithm of the joint likelihood. Which familiar loss has the same minimizers?

Transfer check: change the noise, derive the loss

Assume independent Laplace noise with fixed scale $b > 0$:

$$p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \frac{1}{2b} \exp \left(-\frac{|y_n - \mathbf{x}_n^\top \mathbf{w}|}{b} \right).$$

Take the negative logarithm of the joint likelihood. Which familiar loss has the same minimizers?

$$-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \frac{1}{b} \sum_{n=1}^N |y_n - \mathbf{x}_n^\top \mathbf{w}| + N \log(2b).$$

Transfer check: change the noise, derive the loss

Assume independent Laplace noise with fixed scale $b > 0$:

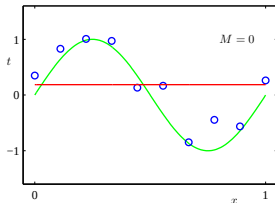
$$p(y_n \mid \mathbf{x}_n, \mathbf{w}) = \frac{1}{2b} \exp \left(-\frac{|y_n - \mathbf{x}_n^\top \mathbf{w}|}{b} \right).$$

Take the negative logarithm of the joint likelihood. Which familiar loss has the same minimizers?

$$-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \frac{1}{b} \sum_{n=1}^N |y_n - \mathbf{x}_n^\top \mathbf{w}| + N \log(2b).$$

Answer: mean absolute error (MAE). Noise $\rightarrow$ likelihood $\rightarrow$ negative logarithm $\rightarrow$ loss. Here b denotes the Laplace scale, not an intercept.

Can a good objective compensate for a limited model family?



Teaching simulation: $y_n = g(x_n) + \epsilon_n$.

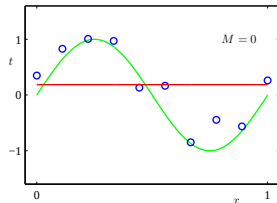
Blue circles: sampled observations.

Red curve: fitted model.

Green curve: generating function g .

The plot's vertical axis t denotes our response y .

Can a good objective compensate for a limited model family?



Teaching simulation: $y_n = g(x_n) + \epsilon_n$.

Blue circles: sampled observations.

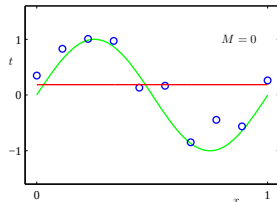
Red curve: fitted model.

Green curve: generating function g .

The plot's vertical axis t denotes our response y .

The degree-zero model predicts a constant. Can changing that constant capture the rise and fall in the data?

Can a good objective compensate for a limited model family?



Teaching simulation: $y_n = g(x_n) + \epsilon_n$.

Blue circles: sampled observations.

Red curve: fitted model.

Green curve: generating function g .

The plot's vertical axis t denotes our response y .

The degree-zero model predicts a constant. Can changing that constant capture the rise and fall in the data?

No. The family lacks flexibility. We know g only because this is a simulation; in practice we observe sampled data.

Mechanism: change the features, keep linear weights

For scalar input x , choose degree M and form a *column* vector:

$$\phi(x) = [1, x, x^2, \dots, x^M]^\top \in \mathbb{R}^{M+1}.$$

Mechanism: change the features, keep linear weights

For scalar input x , choose degree M and form a *column* vector:

$$\phi(x) = [1, x, x^2, \dots, x^M]^\top \in \mathbb{R}^{M+1}.$$

$$f_{\mathbf{w}}(x) = \phi(x)^\top \mathbf{w} = w_0 + w_1x + \dots + w_Mx^M.$$

The function can curve in x while remaining linear in the weights. The constant feature supplies the intercept w_0 .

Mechanism: change the features, keep linear weights

For scalar input x , choose degree M and form a *column* vector:

$$\phi(x) = [1, x, x^2, \dots, x^M]^\top \in \mathbb{R}^{M+1}.$$

$$f_{\mathbf{w}}(x) = \phi(x)^\top \mathbf{w} = w_0 + w_1x + \dots + w_Mx^M.$$

The function can curve in x while remaining linear in the weights. The constant feature supplies the intercept w_0 .

Takeaway

Replace row $\mathbf{x}_n^\top$ of the design by $\phi(x_n)^\top$. We can use the same least-squares solvers to fit the new design.

What can increasing the degree guarantee?

Polynomial families are nested: $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots \subseteq \mathcal{F}_M$. A degree- M polynomial is also degree at most $M + 1$ by setting its new coefficient to zero.

Fit each family by globally minimizing the same training MSE. Can the minimum training error increase when we add a feature? What about error on unseen data?

What can increasing the degree guarantee?

Polynomial families are nested: $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots \subseteq \mathcal{F}_M$. A degree- M polynomial is also degree at most $M + 1$ by setting its new coefficient to zero.

Fit each family by globally minimizing the same training MSE. Can the minimum training error increase when we add a feature? What about error on unseen data?

Training error cannot increase: all previous fits remain available. This statement assumes exact minimization of the unregularized loss.

What can increasing the degree guarantee?

Polynomial families are nested: $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots \subseteq \mathcal{F}_M$. A degree- M polynomial is also degree at most $M + 1$ by setting its new coefficient to zero.

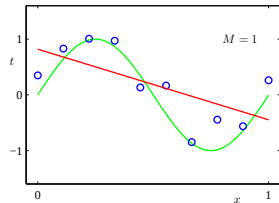
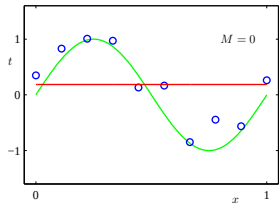
Fit each family by globally minimizing the same training MSE. Can the minimum training error increase when we add a feature? What about error on unseen data?

Training error cannot increase: all previous fits remain available. This statement assumes exact minimization of the unregularized loss.

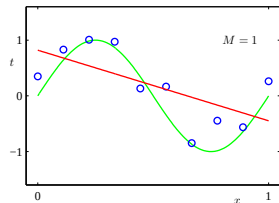
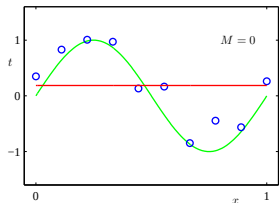
Takeaway

Lower training error alone gives no guarantee of better performance on unseen data. Flexibility can capture sample-specific variation.

Limited flexibility: inspect the training structure

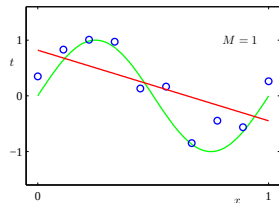
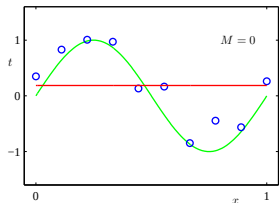


Limited flexibility: inspect the training structure



Observe: neither a constant ($M = 0$) nor a straight line ($M = 1$) captures the rise and fall of these observations.

Limited flexibility: inspect the training structure

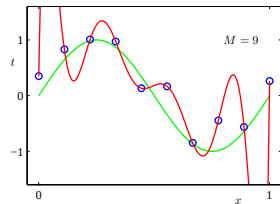
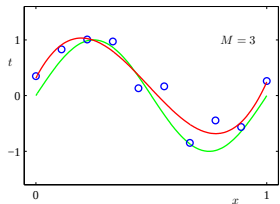


Observe: neither a constant ($M = 0$) nor a straight line ($M = 1$) captures the rise and fall of these observations.

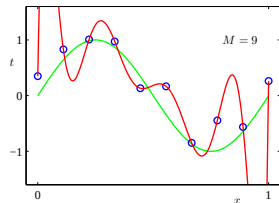
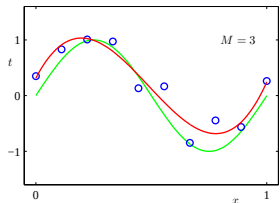
Takeaway

Under-fitting: an inflexible family leaves important training structure unexplained. We expect high error on similar unseen data as well; fitting the same family more accurately will not add curvature.

More flexibility: when the sample is fitted too closely

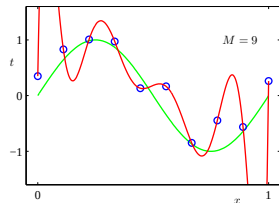
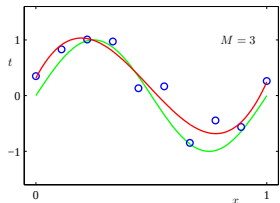


More flexibility: when the sample is fitted too closely



Observe: $M = 3$ follows the broad pattern; $M = 9$ passes through all ten training points and oscillates sharply between them.

More flexibility: when the sample is fitted too closely

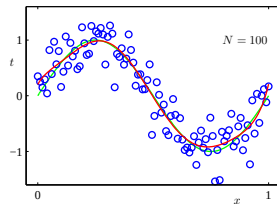
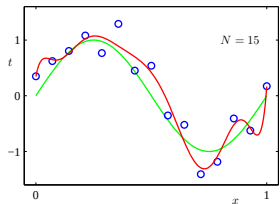


Observe: $M = 3$ follows the broad pattern; $M = 9$ passes through all ten training points and oscillates sharply between them.

Takeaway

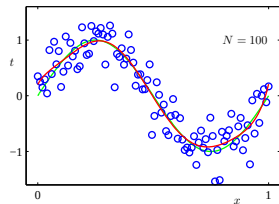
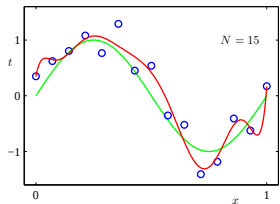
In this simulation, $M = 9$ fits sample-specific noise: its lower training error comes with worse expected unseen-data performance. Interpolation alone is not a universal proof of over-fitting.

Keep the degree fixed, change the amount of evidence



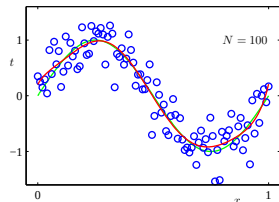
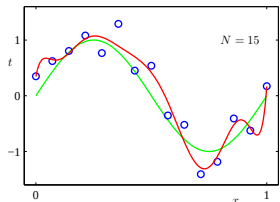
Same degree $M = 9$: compare $N = 15$ and $N = 100$.

Keep the degree fixed, change the amount of evidence



Same degree $M = 9$: compare $N = 15$ and $N = 100$. More representative observations can constrain sample-specific fluctuations. The larger sample here supports a smoother fit.

Keep the degree fixed, change the amount of evidence



Same degree $M = 9$: compare $N = 15$ and $N = 100$. More representative observations can constrain sample-specific fluctuations. The larger sample here supports a smoother fit.

This illustrates a possible benefit, not a guarantee for every sample. If more data are unavailable, how else could we control flexibility? Session 3 introduces regularization.

Define the problem through training and unseen data

Use two pieces of evidence: what remains unexplained in training, and how sensitive is the fitted curve between observations?

Define the problem through training and unseen data

Use two pieces of evidence: what remains unexplained in training, and how sensitive is the fitted curve between observations?

Under-fitting: the family is too inflexible to capture important training structure; training error and expected unseen error are high.

Define the problem through training and unseen data

Use two pieces of evidence: what remains unexplained in training, and how sensitive is the fitted curve between observations?

Under-fitting: the family is too inflexible to capture important training structure; training error and expected unseen error are high.

Over-fitting: the fitted model follows sample-specific variation, achieving low training error but worse expected unseen-data performance.

Define the problem through training and unseen data

Use two pieces of evidence: what remains unexplained in training, and how sensitive is the fitted curve between observations?

Under-fitting: the family is too inflexible to capture important training structure; training error and expected unseen error are high.

Over-fitting: the fitted model follows sample-specific variation, achieving low training error but worse expected unseen-data performance.

Takeaway

In real applications we have sampled data, not the generating curve. We need evidence beyond training error to diagnose generalization. Next lecture introduces validation, model selection, and bias-variance.

A diagnostic grid: what does the training sample tell us?

Inspect evidence before naming the behavior.

A diagnostic grid: what does the training sample tell us?

Inspect evidence before naming the behavior.

Residuals retain systematic structure + limited flexibility $\implies$ an *under-fitting clue*.

A diagnostic grid: what does the training sample tell us?

Inspect evidence before naming the behavior.

Residuals retain systematic structure + limited flexibility $\implies$ an *under-fitting clue*.

Near-zero training error + oscillation or sample sensitivity $\implies$ an *over-fitting clue*.

A diagnostic grid: what does the training sample tell us?

Inspect evidence before naming the behavior.

Residuals retain systematic structure + limited flexibility $\implies$ an *under-fitting* clue.

Near-zero training error + oscillation or sample sensitivity $\implies$ an *over-fitting* clue.

Takeaway

Small training error alone does not establish unseen performance. These observations are diagnostic clues, not a measured test error.

A diagnostic grid: what does the training sample tell us?

Inspect evidence before naming the behavior.

Residuals retain systematic structure + limited flexibility $\implies$ an *under-fitting clue*.

Near-zero training error + oscillation or sample sensitivity $\implies$ an *over-fitting clue*.

Takeaway

Small training error alone does not establish unseen performance. These observations are diagnostic clues, not a measured test error.

Next action: try intermediate complexity or regularization; use validation in Lecture 04 to compare unseen-data estimates.

Session 2: diagnose two fits on a new noisy sample

On the same 20 observations, fit A has degree 1 and MSE 0.40; fit B has degree 19 and MSE 0.00. A leaves curved structure in its residuals; B oscillates between points. Compare flexibility, training fit, and expected unseen-data behavior. What would you try next?

Session 2: diagnose two fits on a new noisy sample

On the same 20 observations, fit A has degree 1 and MSE 0.40; fit B has degree 19 and MSE 0.00. A leaves curved structure in its residuals; B oscillates between points. Compare flexibility, training fit, and expected unseen-data behavior. What would you try next?

A: limited flexibility and missed training structure suggest under-fitting; expect high unseen error.

Session 2: diagnose two fits on a new noisy sample

On the same 20 observations, fit A has degree 1 and MSE 0.40; fit B has degree 19 and MSE 0.00. A leaves curved structure in its residuals; B oscillates between points. Compare flexibility, training fit, and expected unseen-data behavior. What would you try next?

A: limited flexibility and missed training structure suggest under-fitting; expect high unseen error.

B: high flexibility and perfect training fit, with oscillations, suggest over-fitting; expect worse unseen performance than a fit capturing the broad pattern.

Session 2: diagnose two fits on a new noisy sample

On the same 20 observations, fit A has degree 1 and MSE 0.40; fit B has degree 19 and MSE 0.00. A leaves curved structure in its residuals; B oscillates between points. Compare flexibility, training fit, and expected unseen-data behavior. What would you try next?

A: limited flexibility and missed training structure suggest under-fitting; expect high unseen error.

B: high flexibility and perfect training fit, with oscillations, suggest over-fitting; expect worse unseen performance than a fit capturing the broad pattern.

These are diagnostic clues: training error cannot rank A versus B on unseen data. Validation next lecture will provide the missing evidence.

Session 2: diagnose two fits on a new noisy sample

On the same 20 observations, fit A has degree 1 and MSE 0.40; fit B has degree 19 and MSE 0.00. A leaves curved structure in its residuals; B oscillates between points. Compare flexibility, training fit, and expected unseen-data behavior. What would you try next?

A: limited flexibility and missed training structure suggest under-fitting; expect high unseen error.

B: high flexibility and perfect training fit, with oscillations, suggest over-fitting; expect worse unseen performance than a fit capturing the broad pattern.

These are diagnostic clues: training error cannot rank A versus B on unseen data. Validation next lecture will provide the missing evidence.

Decision: consider an intermediate degree or regularization.

Optional: MLE asymptotics

Session 3: How do we control complexity?

A flexible model fits the training sample very well. How can we keep useful features while making its fitted weights less sensitive to this sample?

Regularization: keep the features, change the preference

Polynomial features gave us a richer model family. Large fitted weights can also make predictions sensitive to small changes in the data.

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the observations}} + \underbrace{\lambda R(\mathbf{w})}_{\text{prefer smaller weights}}, \quad \lambda \geq 0.$$

Regularization: keep the features, change the preference

Polynomial features gave us a richer model family. Large fitted weights can also make predictions sensitive to small changes in the data.

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the observations}} + \underbrace{\lambda R(\mathbf{w})}_{\text{prefer smaller weights}}, \quad \lambda \geq 0.$$

A penalty changes which fit we prefer within the same feature family. It can accept more training error in exchange for less sensitivity.

Regularization: keep the features, change the preference

Polynomial features gave us a richer model family. Large fitted weights can also make predictions sensitive to small changes in the data.

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the observations}} + \underbrace{\lambda R(\mathbf{w})}_{\text{prefer smaller weights}}, \quad \lambda \geq 0.$$

A penalty changes which fit we prefer within the same feature family. It can accept more training error in exchange for less sensitivity.

Takeaway

Regularization introduces bias and can reduce variance. Better unseen-data performance is a motivation, not a guarantee.

Before ridge: make coefficients comparable

Use the training data to center and standardize each feature:

$$x_{c,nj} = \frac{x_{nj} - \bar{x}_j}{s_j}, \quad y_{c,n} = y_n - \bar{y},$$

and collect the transformed observations in $\mathbf{X}_c$ and $\mathbf{y}_c$.

Before ridge: make coefficients comparable

Use the training data to center and standardize each feature:

$$x_{c,nj} = \frac{x_{nj} - \bar{x}_j}{s_j}, \quad y_{c,n} = y_n - \bar{y},$$

and collect the transformed observations in $\mathbf{X}_c$ and $\mathbf{y}_c$. Centering handles the baseline separately, so the intercept is not included in the coefficient penalty.

Before ridge: make coefficients comparable

Use the training data to center and standardize each feature:

$$x_{c,nj} = \frac{x_{nj} - \bar{x}_j}{s_j}, \quad y_{c,n} = y_n - \bar{y},$$

and collect the transformed observations in $\mathbf{X}_c$ and $\mathbf{y}_c$. Centering handles the baseline separately, so the intercept is not included in the coefficient penalty.

Takeaway

After standardization, a common λ applies a comparable amount of regularization across features.

L_2 -regularization: the ridge objective

On the centered, standardized design, penalize squared coefficient size:

$$J_\lambda(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \|\mathbf{w}\|_2^2 = \sum_{j=1}^D w_j^2.$$

L_2 -regularization: the ridge objective

On the centered, standardized design, penalize squared coefficient size:

$$J_\lambda(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \|\mathbf{w}\|_2^2 = \sum_{j=1}^D w_j^2.$$

What happens at $\lambda = 0$? What changes as the penalty strengthens?

L_2 -regularization: the ridge objective

On the centered, standardized design, penalize squared coefficient size:

$$J_\lambda(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \|\mathbf{w}\|_2^2 = \sum_{j=1}^D w_j^2.$$

What happens at $\lambda = 0$? What changes as the penalty strengthens?

At $\lambda = 0$ this is ordinary least squares (OLS). Increasing λ puts more weight on shrinking the coefficient vector; individual coordinates need not all change monotonically.

The $1/2$ in each term keeps the derivative simple. The intercept remains outside the penalty.

Explicit ridge solution: follow the factor N

Differentiating the same objective gives

$$\nabla J_{\lambda}(\mathbf{w}) = -\frac{1}{N} \mathbf{X}_c^{\top} (\mathbf{y}_c - \mathbf{X}_c \mathbf{w}) + \lambda \mathbf{w}.$$

Explicit ridge solution: follow the factor N

Differentiating the same objective gives

$$\nabla J_{\lambda}(\mathbf{w}) = -\frac{1}{N}\mathbf{X}_c^{\top}(\mathbf{y}_c - \mathbf{X}_c\mathbf{w}) + \lambda\mathbf{w}.$$

Set the gradient to zero:

$$-\mathbf{X}_c^{\top}(\mathbf{y}_c - \mathbf{X}_c\mathbf{w}) + N\lambda\mathbf{w} = \mathbf{0}.$$

Explicit ridge solution: follow the factor N

Differentiating the same objective gives

$$\nabla J_{\lambda}(\mathbf{w}) = -\frac{1}{N}\mathbf{X}_c^{\top}(\mathbf{y}_c - \mathbf{X}_c\mathbf{w}) + \lambda\mathbf{w}.$$

Set the gradient to zero:

$$-\mathbf{X}_c^{\top}(\mathbf{y}_c - \mathbf{X}_c\mathbf{w}) + N\lambda\mathbf{w} = \mathbf{0}.$$

Collect the terms containing $\mathbf{w}$:

$$(\mathbf{X}_c^{\top}\mathbf{X}_c + N\lambda\mathbf{I})\mathbf{w} = \mathbf{X}_c^{\top}\mathbf{y}_c.$$

Explicit ridge solution: follow the factor N

Differentiating the same objective gives

$$\nabla J_{\lambda}(\mathbf{w}) = -\frac{1}{N}\mathbf{X}_c^{\top}(\mathbf{y}_c - \mathbf{X}_c\mathbf{w}) + \lambda\mathbf{w}.$$

Set the gradient to zero:

$$-\mathbf{X}_c^{\top}(\mathbf{y}_c - \mathbf{X}_c\mathbf{w}) + N\lambda\mathbf{w} = \mathbf{0}.$$

Collect the terms containing $\mathbf{w}$:

$$(\mathbf{X}_c^{\top}\mathbf{X}_c + N\lambda\mathbf{I})\mathbf{w} = \mathbf{X}_c^{\top}\mathbf{y}_c.$$

For $\lambda > 0$ the matrix is invertible:

$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}_c^{\top}\mathbf{X}_c + N\lambda\mathbf{I})^{-1}\mathbf{X}_c^{\top}\mathbf{y}_c.$$

At $\lambda = 0$, this inverse formula reduces to OLS when $\mathbf{X}_c$ has full column rank.

Explicit ridge solution: follow the factor N

Differentiating the same objective gives

$$\nabla J_\lambda(\mathbf{w}) = -\frac{1}{N} \mathbf{X}_c^\top (\mathbf{y}_c - \mathbf{X}_c \mathbf{w}) + \lambda \mathbf{w}.$$

Set the gradient to zero:

$$-\mathbf{X}_c^\top (\mathbf{y}_c - \mathbf{X}_c \mathbf{w}) + N\lambda \mathbf{w} = \mathbf{0}.$$

Collect the terms containing $\mathbf{w}$:

$$(\mathbf{X}_c^\top \mathbf{X}_c + N\lambda \mathbf{I}) \mathbf{w} = \mathbf{X}_c^\top \mathbf{y}_c.$$

For $\lambda > 0$ the matrix is invertible:

$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}_c^\top \mathbf{X}_c + N\lambda \mathbf{I})^{-1} \mathbf{X}_c^\top \mathbf{y}_c.$$

At $\lambda = 0$, this inverse formula reduces to OLS when $\mathbf{X}_c$ has full column rank.

In code: use a linear-system solver (or augmented QR/SVD); the inverse above is mathematics, not an instruction to form it.

Ridge shrinks uncertain directions more

Example. With two almost duplicate sensors, the data may reveal their combined effect but not how to split the weight between them.

Ridge shrinks uncertain directions more

Example. With two almost duplicate sensors, the data may reveal their combined effect but not how to split the weight between them.

Rotate coefficient space to directions $\mathbf{v}_j$ satisfying $\mathbf{X}_c^\top \mathbf{X}_c \mathbf{v}_j = d_j \mathbf{v}_j$ and $\|\mathbf{v}_j\|_2 = 1$.

$$\|\mathbf{X}_c \mathbf{v}_j\|_2^2 = d_j.$$

Large d_j : moving in this direction visibly changes predictions. Small d_j : the data barely distinguish such changes.

Ridge shrinks uncertain directions more

Example. With two almost duplicate sensors, the data may reveal their combined effect but not how to split the weight between them.

Rotate coefficient space to directions $\mathbf{v}_j$ satisfying $\mathbf{X}_c^\top \mathbf{X}_c \mathbf{v}_j = d_j \mathbf{v}_j$ and $\|\mathbf{v}_j\|_2 = 1$.

$$\|\mathbf{X}_c \mathbf{v}_j\|_2^2 = d_j.$$

Large d_j : moving in this direction visibly changes predictions. Small d_j : the data barely distinguish such changes.

For $d_j > 0$, ridge changes the OLS component by

$$\mathbf{v}_j^\top \mathbf{w}_{\text{ridge}} = \underbrace{\frac{d_j}{d_j + N\lambda}}_{\text{shrinkage factor}} \mathbf{v}_j^\top \mathbf{w}_{\text{OLS}}.$$

Ridge shrinks uncertain directions more

Example. With two almost duplicate sensors, the data may reveal their combined effect but not how to split the weight between them.

Rotate coefficient space to directions $\mathbf{v}_j$ satisfying $\mathbf{X}_c^\top \mathbf{X}_c \mathbf{v}_j = d_j \mathbf{v}_j$ and $\|\mathbf{v}_j\|_2 = 1$.

$$\|\mathbf{X}_c \mathbf{v}_j\|_2^2 = d_j.$$

Large d_j : moving in this direction visibly changes predictions. Small d_j : the data barely distinguish such changes.

For $d_j > 0$, ridge changes the OLS component by

$$\mathbf{v}_j^\top \mathbf{w}_{\text{ridge}} = \underbrace{\frac{d_j}{d_j + N\lambda}}_{\text{shrinkage factor}} \mathbf{v}_j^\top \mathbf{w}_{\text{OLS}}.$$

large $d_j \Rightarrow$ factor near 1 $\Rightarrow$ little shrinkage | small $d_j \Rightarrow$ factor near 0 $\Rightarrow$ strong shrinkage

Ridge shrinks uncertain directions more

Example. With two almost duplicate sensors, the data may reveal their combined effect but not how to split the weight between them.

Rotate coefficient space to directions $\mathbf{v}_j$ satisfying $\mathbf{X}_c^\top \mathbf{X}_c \mathbf{v}_j = d_j \mathbf{v}_j$ and $\|\mathbf{v}_j\|_2 = 1$.

$$\|\mathbf{X}_c \mathbf{v}_j\|_2^2 = d_j.$$

Large d_j : moving in this direction visibly changes predictions. Small d_j : the data barely distinguish such changes.

For $d_j > 0$, ridge changes the OLS component by

$$\mathbf{v}_j^\top \mathbf{w}_{\text{ridge}} = \underbrace{\frac{d_j}{d_j + N\lambda}}_{\text{shrinkage factor}} \mathbf{v}_j^\top \mathbf{w}_{\text{OLS}}.$$

large $d_j \Rightarrow$ factor near 1 $\Rightarrow$ little shrinkage | small $d_j \Rightarrow$ factor near 0 $\Rightarrow$ strong shrinkage

Interpretation. Ridge trusts directions the data determine clearly and damps ambiguous ones, such as the difference between duplicate sensors. If $d_j = 0$, it sets that invisible component to zero.

Ridge lifts the small eigenvalues

Write the symmetric Gram matrix as $\mathbf{X}_c^\top \mathbf{X}_c = \mathbf{U} \text{diag}(d_1, \dots, d_D) \mathbf{U}^\top$, with $d_j \geq 0$ and orthogonal $\mathbf{U}$.

Ridge lifts the small eigenvalues

Write the symmetric Gram matrix as $\mathbf{X}_c^\top \mathbf{X}_c = \mathbf{U} \text{diag}(d_1, \dots, d_D) \mathbf{U}^\top$, with $d_j \geq 0$ and orthogonal $\mathbf{U}$. Adding $N\lambda \mathbf{I}$ shifts every eigenvalue by the same amount:

$$\mathbf{X}_c^\top \mathbf{X}_c + N\lambda \mathbf{I} = \mathbf{U} \text{diag}(d_1 + N\lambda, \dots, d_D + N\lambda) \mathbf{U}^\top.$$

Ridge lifts the small eigenvalues

Write the symmetric Gram matrix as $\mathbf{X}_c^\top \mathbf{X}_c = \mathbf{U} \text{diag}(d_1, \dots, d_D) \mathbf{U}^\top$, with $d_j \geq 0$ and orthogonal $\mathbf{U}$. Adding $N\lambda \mathbf{I}$ shifts every eigenvalue by the same amount:

$$\mathbf{X}_c^\top \mathbf{X}_c + N\lambda \mathbf{I} = \mathbf{U} \text{diag}(d_1 + N\lambda, \dots, d_D + N\lambda) \mathbf{U}^\top.$$

For $\lambda > 0$, every eigenvalue is at least $N\lambda > 0$. Even an exactly dependent design now has a unique ridge solution.

$$\kappa_2(\mathbf{X}_c^\top \mathbf{X}_c + N\lambda \mathbf{I}) = \frac{d_{\max} + N\lambda}{d_{\min} + N\lambda}.$$

Ridge lifts the small eigenvalues

Write the symmetric Gram matrix as $\mathbf{X}_c^\top \mathbf{X}_c = \mathbf{U} \text{diag}(d_1, \dots, d_D) \mathbf{U}^\top$, with $d_j \geq 0$ and orthogonal $\mathbf{U}$. Adding $N\lambda \mathbf{I}$ shifts every eigenvalue by the same amount:

$$\mathbf{X}_c^\top \mathbf{X}_c + N\lambda \mathbf{I} = \mathbf{U} \text{diag}(d_1 + N\lambda, \dots, d_D + N\lambda) \mathbf{U}^\top.$$

For $\lambda > 0$, every eigenvalue is at least $N\lambda > 0$. Even an exactly dependent design now has a unique ridge solution.

$$\kappa_2(\mathbf{X}_c^\top \mathbf{X}_c + N\lambda \mathbf{I}) = \frac{d_{\max} + N\lambda}{d_{\min} + N\lambda}.$$

Takeaway

The lift reduces unequal eigenvalue ratios and limits sensitivity along weakly supported directions. It also changes the fit.

Check: two nearly redundant sensors

Two standardized sensor features are almost identical. Their Gram eigenvalues are 199.98 and 0.02. If $N = 100$ and $\lambda = 0.1$, predict the new condition number. Why might ridge help? What does it cost?

Check: two nearly redundant sensors

Two standardized sensor features are almost identical. Their Gram eigenvalues are 199.98 and 0.02. If $N = 100$ and $\lambda = 0.1$, predict the new condition number. Why might ridge help? What does it cost?

$$\kappa_{\text{OLS}} = \frac{199.98}{0.02} = 9,999, \quad \kappa_{\text{ridge}} = \frac{199.98 + 10}{0.02 + 10} \approx 20.96.$$

The system is much less sensitive in the nearly redundant direction.

Check: two nearly redundant sensors

Two standardized sensor features are almost identical. Their Gram eigenvalues are 199.98 and 0.02. If $N = 100$ and $\lambda = 0.1$, predict the new condition number. Why might ridge help? What does it cost?

$$\kappa_{\text{OLS}} = \frac{199.98}{0.02} = 9,999, \quad \kappa_{\text{ridge}} = \frac{199.98 + 10}{0.02 + 10} \approx 20.96.$$

The system is much less sensitive in the nearly redundant direction. **Decision:** ridge is a plausible choice if both sensors carry useful signal. Shrinkage trades some fit and bias for stability; this calculation alone does not establish better unseen predictions.

L_1 -regularization: the lasso

Suppose a small subset of the available features plausibly carries the signal.

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_1, \quad \|\mathbf{w}\|_1 = \sum_{j=1}^D |w_j|.$$

L_1 -regularization: the lasso

Suppose a small subset of the available features plausibly carries the signal.

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_1, \quad \|\mathbf{w}\|_1 = \sum_{j=1}^D |w_j|.$$

The data-fit term is still squared error. The change from ridge is the penalty: absolute values instead of squares.

$|w_j|$ has a corner at zero. This makes exact-zero coefficients common for suitable data and penalty strengths.

L_1 -regularization: the lasso

Suppose a small subset of the available features plausibly carries the signal.

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_1, \quad \|\mathbf{w}\|_1 = \sum_{j=1}^D |w_j|.$$

The data-fit term is still squared error. The change from ridge is the penalty: absolute values instead of squares.

$|w_j|$ has a corner at zero. This makes exact-zero coefficients common for suitable data and penalty strengths.

Takeaway

Lasso can produce a sparse, interpretable model. It does not guarantee exact recovery of the true relevant features. Features are standardized; the intercept is not penalized.

One-dimensional lasso: when does a coefficient become zero?

For one normalized coordinate, consider

$$q(w) = \frac{1}{2}(w - z)^2 + \lambda|w|, \quad \lambda \geq 0.$$

Think of z as the unregularized coefficient. What should happen when its magnitude is no larger than the penalty threshold λ ?

One-dimensional lasso: when does a coefficient become zero?

For one normalized coordinate, consider

$$q(w) = \frac{1}{2}(w - z)^2 + \lambda|w|, \quad \lambda \geq 0.$$

Think of z as the unregularized coefficient. What should happen when its magnitude is no larger than the penalty threshold λ ?

If $w > 0$, $q'(w) = w - z + \lambda = 0$, giving $w = z - \lambda$ when $z > \lambda$.

One-dimensional lasso: when does a coefficient become zero?

For one normalized coordinate, consider

$$q(w) = \frac{1}{2}(w - z)^2 + \lambda|w|, \quad \lambda \geq 0.$$

Think of z as the unregularized coefficient. What should happen when its magnitude is no larger than the penalty threshold λ ?

If $w > 0$, $q'(w) = w - z + \lambda = 0$, giving $w = z - \lambda$ when $z > \lambda$. If $w < 0$, $q'(w) = w - z - \lambda = 0$, giving $w = z + \lambda$ when $z < -\lambda$.

One-dimensional lasso: when does a coefficient become zero?

For one normalized coordinate, consider

$$q(w) = \frac{1}{2}(w - z)^2 + \lambda|w|, \quad \lambda \geq 0.$$

Think of z as the unregularized coefficient. What should happen when its magnitude is no larger than the penalty threshold λ ?

If $w > 0$, $q'(w) = w - z + \lambda = 0$, giving $w = z - \lambda$ when $z > \lambda$. If $w < 0$, $q'(w) = w - z - \lambda = 0$, giving $w = z + \lambda$ when $z < -\lambda$. At the corner $w = 0$, optimality holds when $|z| \leq \lambda$.

One-dimensional lasso: when does a coefficient become zero?

For one normalized coordinate, consider

$$q(w) = \frac{1}{2}(w - z)^2 + \lambda|w|, \quad \lambda \geq 0.$$

Think of z as the unregularized coefficient. What should happen when its magnitude is no larger than the penalty threshold λ ?

If $w > 0$, $q'(w) = w - z + \lambda = 0$, giving $w = z - \lambda$ when $z > \lambda$. If $w < 0$, $q'(w) = w - z - \lambda = 0$, giving $w = z + \lambda$ when $z < -\lambda$. At the corner $w = 0$, optimality holds when $|z| \leq \lambda$.

$$S_\lambda(z) = \text{sign}(z)(|z| - \lambda)_+ = \begin{cases} z - \lambda, & z > \lambda, \\ 0, & |z| \leq \lambda, \\ z + \lambda, & z < -\lambda. \end{cases}$$

One-dimensional lasso: when does a coefficient become zero?

For one normalized coordinate, consider

$$q(w) = \frac{1}{2}(w - z)^2 + \lambda|w|, \quad \lambda \geq 0.$$

Think of z as the unregularized coefficient. What should happen when its magnitude is no larger than the penalty threshold λ ?

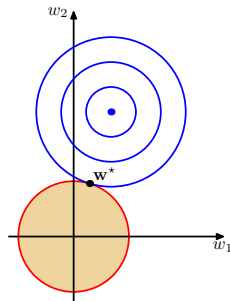
If $w > 0$, $q'(w) = w - z + \lambda = 0$, giving $w = z - \lambda$ when $z > \lambda$. If $w < 0$, $q'(w) = w - z - \lambda = 0$, giving $w = z + \lambda$ when $z < -\lambda$. At the corner $w = 0$, optimality holds when $|z| \leq \lambda$.

$$S_\lambda(z) = \text{sign}(z)(|z| - \lambda)_+ = \begin{cases} z - \lambda, & z > \lambda, \\ 0, & |z| \leq \lambda, \\ z + \lambda, & z < -\lambda. \end{cases}$$

This is exact for the stated one-dimensional (or orthonormal-coordinate) problem. Correlated multivariate designs do not share this one-shot formula.

Ridge geometry: a budget on squared coefficient size

Write $\mathcal{L}_c(\mathbf{w}) = \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 / (2N)$.

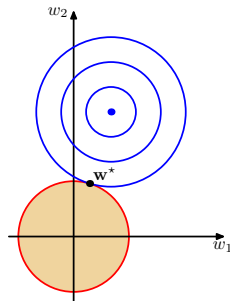


$$\min_{\mathbf{w}} \mathcal{L}_c(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_2^2 \leq t.$$

Blue curves: equal unpenalized loss. The disk limits the weights.

Ridge geometry: a budget on squared coefficient size

Write $\mathcal{L}_c(\mathbf{w}) = \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 / (2N)$.



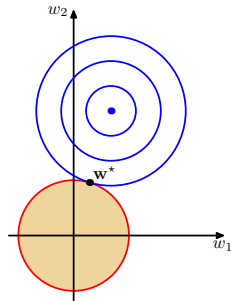
Blue curves: equal unpenalized loss. The disk limits the weights.

$$\min_{\mathbf{w}} \mathcal{L}_c(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_2^2 \leq t.$$

Find the lowest loss contour that meets the feasible disk.

Ridge geometry: a budget on squared coefficient size

Write $\mathcal{L}_c(\mathbf{w}) = \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 / (2N)$.



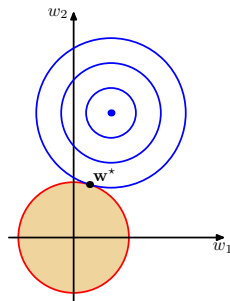
Blue curves: equal unpenalized loss. The disk limits the weights.

$$\min_{\mathbf{w}} \mathcal{L}_c(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_2^2 \leq t.$$

Find the lowest loss contour that meets the feasible disk. When the constraint is active, contact is on the boundary. An unconstrained optimum inside the disk remains feasible.

Ridge geometry: a budget on squared coefficient size

Write $\mathcal{L}_c(\mathbf{w}) = \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 / (2N)$.



Blue curves: equal unpenalized loss. The disk limits the weights.

$$\min_{\mathbf{w}} \mathcal{L}_c(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_2^2 \leq t.$$

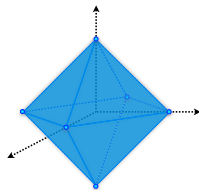
Find the lowest loss contour that meets the feasible disk. When the constraint is active, contact is on the boundary. An unconstrained optimum inside the disk remains feasible.

Each penalized solution has a corresponding budget $t \geq 0$. Here t bounds squared norm; the disk radius is $\sqrt{t}$.

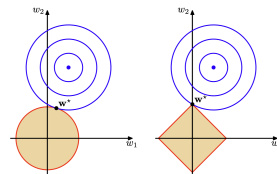
t and λ are not the same numerical parameter.
Stronger penalties generally mean tighter budgets.

Lasso geometry: corners lie on coordinate axes

$$\min_{\mathbf{w}} \mathcal{L}_c(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_1 \leq t.$$



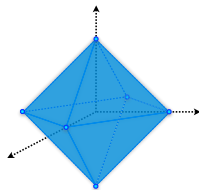
A three-dimensional L_1 ball.



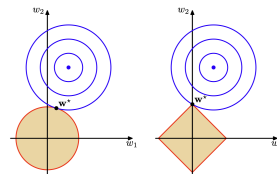
Smooth L_2 boundary versus L_1 corners.

Lasso geometry: corners lie on coordinate axes

$$\min_{\mathbf{w}} \mathcal{L}_c(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_1 \leq t.$$



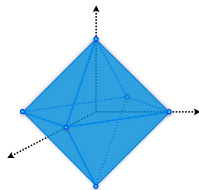
A three-dimensional L_1 ball.
Imagine expanding a loss contour until it first touches the feasible set.



Smooth L_2 boundary versus L_1 corners.

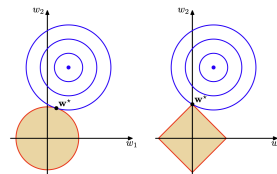
Lasso geometry: corners lie on coordinate axes

$$\min_{\mathbf{w}} \mathcal{L}_c(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_1 \leq t.$$



A three-dimensional L_1 ball.

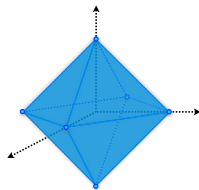
Imagine expanding a loss contour until it first touches the feasible set. In two dimensions, that first contact may be a corner: one coefficient is then exactly zero. Contact along an edge is also possible.



Smooth L_2 boundary versus L_1 corners.

Lasso geometry: corners lie on coordinate axes

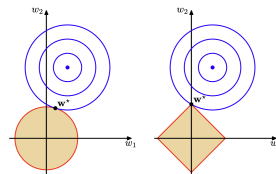
$$\min_{\mathbf{w}} \mathcal{L}_c(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_1 \leq t.$$



A three-dimensional L_1 ball.

Imagine expanding a loss contour until it first touches the feasible set. In two dimensions, that first contact may be a corner: one coefficient is then exactly zero. Contact along an edge is also possible.

Exact zeros are common, not guaranteed for every dataset and penalty. The budget t corresponds to a penalized solution; it is not λ .



Smooth L_2 boundary versus L_1 corners.

Check: does a zero coefficient identify the truth?

Two predictors carry nearly the same information. Lasso keeps one and gives the other coefficient zero. Does that prove the discarded predictor has no predictive value?

Check: does a zero coefficient identify the truth?

Two predictors carry nearly the same information. Lasso keeps one and gives the other coefficient zero. Does that prove the discarded predictor has no predictive value?

No. Either predictor may explain much of the same signal. A sparse fit can choose one representative; the selected feature may change with the sample.

Check: does a zero coefficient identify the truth?

Two predictors carry nearly the same information. Lasso keeps one and gives the other coefficient zero. Does that prove the discarded predictor has no predictive value?

No. Either predictor may explain much of the same signal. A sparse fit can choose one representative; the selected feature may change with the sample.

Takeaway

Prefer lasso when sparsity is a defensible modeling assumption. For many correlated predictors with dense signal, ridge is often a more natural starting point.

From likelihood to MAP: add a belief about weights

Start with $y_n = b + \mathbf{x}_n^\top \mathbf{w} + \epsilon_n$, conditional on the inputs:

$$\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \quad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I}), \quad \sigma^2, \tau^2 > 0.$$

From likelihood to MAP: add a belief about weights

Start with $y_n = b + \mathbf{x}_n^\top \mathbf{w} + \epsilon_n$, conditional on the inputs:

$$\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \quad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I}), \quad \sigma^2, \tau^2 > 0.$$

Use a flat prior for b ; optimizing it gives $b = \bar{y} - \bar{\mathbf{x}}^\top \mathbf{w}$ and the centered residual.

From likelihood to MAP: add a belief about weights

Start with $y_n = b + \mathbf{x}_n^\top \mathbf{w} + \epsilon_n$, conditional on the inputs:

$$\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \quad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I}), \quad \sigma^2, \tau^2 > 0.$$

Use a flat prior for b ; optimizing it gives $b = \bar{y} - \bar{\mathbf{x}}^\top \mathbf{w}$ and the centered residual.

Likelihood: fit the data

$$-\log p(\mathbf{y} \mid \mathbf{X}, b, \mathbf{w}) = \frac{\|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2}{2\sigma^2} + C$$

evaluated at the optimal intercept.

From likelihood to MAP: add a belief about weights

Start with $y_n = b + \mathbf{x}_n^\top \mathbf{w} + \epsilon_n$, conditional on the inputs:

$$\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \quad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I}), \quad \sigma^2, \tau^2 > 0.$$

Use a flat prior for b ; optimizing it gives $b = \bar{y} - \bar{\mathbf{x}}^\top \mathbf{w}$ and the centered residual.

Likelihood: fit the data

$$-\log p(\mathbf{y} \mid \mathbf{X}, b, \mathbf{w}) = \frac{\|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2}{2\sigma^2} + C$$

evaluated at the optimal intercept.

Prior: prefer smaller weights

$$-\log p(\mathbf{w}) = \frac{\|\mathbf{w}\|_2^2}{2\tau^2} + C'$$

Smaller τ^2 expresses a stronger preference for zero.

MAP becomes ridge with a specific penalty strength

Bayes' rule: $p(b, \mathbf{w} \mid \mathbf{X}, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{X}, b, \mathbf{w}) p(\mathbf{w})$ with a flat intercept prior; optimize over b as on the previous frame.

MAP becomes ridge with a specific penalty strength

Bayes' rule: $p(b, \mathbf{w} \mid \mathbf{X}, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{X}, b, \mathbf{w}) p(\mathbf{w})$ with a flat intercept prior; optimize over b as on the previous frame.

$$-\log p(b, \mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \underbrace{\frac{\|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2}{2\sigma^2}}_{\text{likelihood}} + \underbrace{\frac{\|\mathbf{w}\|_2^2}{2\tau^2}}_{\text{prior}} + C''.$$

MAP becomes ridge with a specific penalty strength

Bayes' rule: $p(b, \mathbf{w} \mid \mathbf{X}, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{X}, b, \mathbf{w}) p(\mathbf{w})$ with a flat intercept prior; optimize over b as on the previous frame.

$$-\log p(b, \mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \underbrace{\frac{\|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2}{2\sigma^2}}_{\text{likelihood}} + \underbrace{\frac{\|\mathbf{w}\|_2^2}{2\tau^2}}_{\text{prior}} + C''.$$

Divide by $N/\sigma^2 > 0$: the minimizer is unchanged.

$$\frac{1}{2N} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 + \frac{\sigma^2}{2N\tau^2} \|\mathbf{w}\|_2^2 = J_\lambda(\mathbf{w}), \quad \boxed{\lambda = \frac{\sigma^2}{N\tau^2}}.$$

MAP becomes ridge with a specific penalty strength

Bayes' rule: $p(b, \mathbf{w} \mid \mathbf{X}, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{X}, b, \mathbf{w}) p(\mathbf{w})$ with a flat intercept prior; optimize over b as on the previous frame.

$$-\log p(b, \mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \underbrace{\frac{\|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2}{2\sigma^2}}_{\text{likelihood}} + \underbrace{\frac{\|\mathbf{w}\|_2^2}{2\tau^2}}_{\text{prior}} + C''.$$

Divide by $N/\sigma^2 > 0$: the minimizer is unchanged.

$$\frac{1}{2N} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{w}\|_2^2 + \frac{\sigma^2}{2N\tau^2} \|\mathbf{w}\|_2^2 = J_\lambda(\mathbf{w}), \quad \boxed{\lambda = \frac{\sigma^2}{N\tau^2}}.$$

Takeaway

Gaussian noise plus a Gaussian weight prior gives ridge MAP. For fixed N , more noise or a tighter prior means stronger shrinkage. No penalty is applied to the intercept.

Session 3: choose a method for a new problem

Choose OLS, ridge, or lasso for each case. Give a reason tied to conditioning, shrinkage/generalization, or sparsity.

- 1 Six well-conditioned predictors; ample data and no reason to shrink.
- 2 Many correlated predictors; small effects spread across most of them.
- 3 Hundreds of candidate predictors; domain knowledge supports a small subset.

Session 3: choose a method for a new problem

Choose OLS, ridge, or lasso for each case. Give a reason tied to conditioning, shrinkage/generalization, or sparsity.

- 1 Six well-conditioned predictors; ample data and no reason to shrink.
- 2 Many correlated predictors; small effects spread across most of them.
- 3 Hundreds of candidate predictors; domain knowledge supports a small subset.

1. **OLS:** no substantive shrinkage motivation, stable design.

Session 3: choose a method for a new problem

Choose OLS, ridge, or lasso for each case. Give a reason tied to conditioning, shrinkage/generalization, or sparsity.

- 1 Six well-conditioned predictors; ample data and no reason to shrink.
- 2 Many correlated predictors; small effects spread across most of them.
- 3 Hundreds of candidate predictors; domain knowledge supports a small subset.

1. OLS: no substantive shrinkage motivation, stable design.

2. Ridge: dense signal; shrinkage stabilizes correlated directions.

Session 3: choose a method for a new problem

Choose OLS, ridge, or lasso for each case. Give a reason tied to conditioning, shrinkage/generalization, or sparsity.

- 1 Six well-conditioned predictors; ample data and no reason to shrink.
- 2 Many correlated predictors; small effects spread across most of them.
- 3 Hundreds of candidate predictors; domain knowledge supports a small subset.

1. **OLS:** no substantive shrinkage motivation, stable design.
2. **Ridge:** dense signal; shrinkage stabilizes correlated directions.
3. **Lasso:** a sparse signal assumption supports feature selection.

Session 3: choose a method for a new problem

Choose OLS, ridge, or lasso for each case. Give a reason tied to conditioning, shrinkage/generalization, or sparsity.

- 1 Six well-conditioned predictors; ample data and no reason to shrink.
- 2 Many correlated predictors; small effects spread across most of them.
- 3 Hundreds of candidate predictors; domain knowledge supports a small subset.

1. **OLS:** no substantive shrinkage motivation, stable design.

2. **Ridge:** dense signal; shrinkage stabilizes correlated directions.

3. **Lasso:** a sparse signal assumption supports feature selection.

A complete answer: name the method and justify its assumption. Standardize features before penalties; keep the intercept unpenalized. Select λ and compare unseen performance through validation in Lecture 04 (next lecture), alongside generalization and bias–variance.

Optional: Rayleigh proof

Deepening: a Gaussian vector

OPTIONAL

For $\mathbf{y} \in \mathbb{R}^N$, the ordinary multivariate Gaussian density requires a *positive definite* covariance Σ :

$$p(\mathbf{y}) = \frac{\exp \left[-\frac{1}{2} (\mathbf{y} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{y} - \boldsymbol{\mu}) \right]}{(2\pi)^{N/2} \det(\Sigma)^{1/2}}.$$

[Return to Session 2 check](#)

Deepening: a Gaussian vector

OPTIONAL

For $\mathbf{y} \in \mathbb{R}^N$, the ordinary multivariate Gaussian density requires a *positive definite* covariance Σ :

$$p(\mathbf{y}) = \frac{\exp \left[-\frac{1}{2} (\mathbf{y} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{y} - \boldsymbol{\mu}) \right]}{(2\pi)^{N/2} \det(\Sigma)^{1/2}}.$$

In our conditional model, $\boldsymbol{\mu} = \mathbf{X}\mathbf{w}$ and $\Sigma = \sigma^2 \mathbf{I}$ with $\sigma^2 > 0$. The density then factors into the N scalar densities used in Session 2.

A singular positive semidefinite covariance defines a degenerate Gaussian, but the inverse/determinant density above does not apply.

[Return to Session 2 check](#)

Optional: MLE in large samples

OPTIONAL

For iid observations, the *average* log-likelihood estimates an expectation:

$$\frac{1}{N} \sum_{n=1}^N \log p(y_n \mid \mathbf{x}_n, \mathbf{w}) \longrightarrow \mathbb{E}_{x,y}[\log p(y \mid x, \mathbf{w})].$$

Suppose the model is correctly specified and identifiable; $\mathbf{w}_{\text{true}}$ is an interior point; a uniform law of large numbers and regular, integrable derivatives apply; and the information below is nonsingular. Then a consistent MLE obeys

$$\mathbf{w}_{\text{MLE}} \xrightarrow{p} \mathbf{w}_{\text{true}}, \quad \sqrt{N}(\mathbf{w}_{\text{MLE}} - \mathbf{w}_{\text{true}}) \xrightarrow{d} \mathcal{N}(\mathbf{0}, \mathbf{I}_1(\mathbf{w}_{\text{true}})^{-1}).$$

[Return to Session 2 check](#)

Optional: MLE in large samples

OPTIONAL

For iid observations, the *average* log-likelihood estimates an expectation:

$$\frac{1}{N} \sum_{n=1}^N \log p(y_n \mid \mathbf{x}_n, \mathbf{w}) \longrightarrow \mathbb{E}_{x,y}[\log p(y \mid x, \mathbf{w})].$$

Suppose the model is correctly specified and identifiable; $\mathbf{w}_{\text{true}}$ is an interior point; a uniform law of large numbers and regular, integrable derivatives apply; and the information below is nonsingular. Then a consistent MLE obeys

$$\mathbf{w}_{\text{MLE}} \xrightarrow{p} \mathbf{w}_{\text{true}}, \quad \sqrt{N}(\mathbf{w}_{\text{MLE}} - \mathbf{w}_{\text{true}}) \xrightarrow{d} \mathcal{N}(\mathbf{0}, \mathbf{I}_1(\mathbf{w}_{\text{true}})^{-1}).$$

$\mathbf{I}_1(\mathbf{w}) = \mathbb{E}[-\nabla_{\mathbf{w}}^2 \log p(y \mid x, \mathbf{w})]$ is per-observation Fisher information. Under the same regularity, it equals the score covariance. Asymptotically, covariance is approximately $\mathbf{I}_1^{-1}/N$; efficiency is among regular estimators and is not a finite-sample equality.

[Return to Session 2 check](#)

Deepening: two other ways to see convexity

OPTIONAL

Composition argument. Each $\mathcal{L}_n(\mathbf{w}) = \frac{1}{2}(y_n - \mathbf{x}_n^\top \mathbf{w})^2$ is a convex square composed with an affine function. A nonnegative average of convex functions remains convex.

[Return to Session 1 check](#)

Deepening: two other ways to see convexity

OPTIONAL

Composition argument. Each $\mathcal{L}_n(\mathbf{w}) = \frac{1}{2}(y_n - \mathbf{x}_n^\top \mathbf{w})^2$ is a convex square composed with an affine function. A nonnegative average of convex functions remains convex.

Definition argument. For $\lambda \in [0, 1]$,

$$\begin{aligned} \mathcal{L}(\lambda \mathbf{w} + (1 - \lambda) \mathbf{w}') - \lambda \mathcal{L}(\mathbf{w}) - (1 - \lambda) \mathcal{L}(\mathbf{w}') \\ = -\frac{\lambda(1 - \lambda)}{2N} \|\mathbf{X}(\mathbf{w} - \mathbf{w}')\|_2^2 \leq 0. \end{aligned}$$

Expand the squares to verify the identity. Equality along a nonzero direction is possible if that direction is in the null space of $\mathbf{X}$.

[Return to Session 1 check](#)

Optional: arbitrary-sampling SGD

OPTIONAL

Let $f(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N f_i(\mathbf{w})$. Sample I with probabilities $p_i > 0$ (not necessarily uniform). The importance-weighted estimate at the *current* iterate is

$$\hat{\mathbf{g}}_I(\mathbf{w}^{(t)}) = \frac{1}{Np_I} \nabla f_I(\mathbf{w}^{(t)}).$$

[Return to Session 1 check](#)

Optional: arbitrary-sampling SGD

OPTIONAL

Let $f(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N f_i(\mathbf{w})$. Sample I with probabilities $p_i > 0$ (not necessarily uniform). The importance-weighted estimate at the *current* iterate is

$$\hat{\mathbf{g}}_I(\mathbf{w}^{(t)}) = \frac{1}{Np_I} \nabla f_I(\mathbf{w}^{(t)}).$$

$$\mathbb{E}_I[\hat{\mathbf{g}}_I(\mathbf{w}^{(t)}) \mid \mathbf{w}^{(t)}] = \sum_{i=1}^N p_i \frac{\nabla f_i(\mathbf{w}^{(t)})}{Np_i} = \nabla f(\mathbf{w}^{(t)}).$$

For m iid draws, average these m weighted estimates. The update is $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \hat{\mathbf{g}}(\mathbf{w}^{(t)})$.

[Return to Session 1 check](#)

Optional: arbitrary-sampling SGD

OPTIONAL

Let $f(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N f_i(\mathbf{w})$. Sample I with probabilities $p_i > 0$ (not necessarily uniform). The importance-weighted estimate at the *current* iterate is

$$\hat{\mathbf{g}}_I(\mathbf{w}^{(t)}) = \frac{1}{Np_I} \nabla f_I(\mathbf{w}^{(t)}).$$

$$\mathbb{E}_I[\hat{\mathbf{g}}_I(\mathbf{w}^{(t)}) \mid \mathbf{w}^{(t)}] = \sum_{i=1}^N p_i \frac{\nabla f_i(\mathbf{w}^{(t)})}{Np_i} = \nabla f(\mathbf{w}^{(t)}).$$

For m iid draws, average these m weighted estimates. The update is $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \hat{\mathbf{g}}(\mathbf{w}^{(t)})$. Uniform sampling has $p_i = 1/N$, recovering ∇f_I . Non-uniform sampling needs the $1/(Np_i)$ factor to stay unbiased.

[Return to Session 1 check](#)

Optional: Rayleigh proof of ridge invertibility

OPTIONAL

For a symmetric matrix $\mathbf{A}$ and $\mathbf{v} \neq \mathbf{0}$, define

$$R(\mathbf{A}, \mathbf{v}) = \frac{\mathbf{v}^\top \mathbf{A} \mathbf{v}}{\mathbf{v}^\top \mathbf{v}}, \quad \lambda_{\min}(\mathbf{A}) = \min_{\mathbf{v} \neq \mathbf{0}} R(\mathbf{A}, \mathbf{v}).$$

With $\mathbf{A} = \mathbf{X}^\top \mathbf{X} + N\lambda \mathbf{I}$ and $\lambda > 0$,

$$R(\mathbf{A}, \mathbf{v}) = \frac{\|\mathbf{X}\mathbf{v}\|_2^2 + N\lambda\|\mathbf{v}\|_2^2}{\|\mathbf{v}\|_2^2} \geq N\lambda.$$

[Return to Session 3 check](#)

Optional: Rayleigh proof of ridge invertibility

OPTIONAL

For a symmetric matrix $\mathbf{A}$ and $\mathbf{v} \neq \mathbf{0}$, define

$$R(\mathbf{A}, \mathbf{v}) = \frac{\mathbf{v}^\top \mathbf{A} \mathbf{v}}{\mathbf{v}^\top \mathbf{v}}, \quad \lambda_{\min}(\mathbf{A}) = \min_{\mathbf{v} \neq \mathbf{0}} R(\mathbf{A}, \mathbf{v}).$$

With $\mathbf{A} = \mathbf{X}^\top \mathbf{X} + N\lambda \mathbf{I}$ and $\lambda > 0$,

$$R(\mathbf{A}, \mathbf{v}) = \frac{\|\mathbf{X}\mathbf{v}\|_2^2 + N\lambda\|\mathbf{v}\|_2^2}{\|\mathbf{v}\|_2^2} \geq N\lambda.$$

Therefore $\lambda_{\min}(\mathbf{A}) \geq N\lambda > 0$: the ridge system is positive definite and invertible, even when $\mathbf{X}$ is rank-deficient. This is the Rayleigh-quotient version of eigenvalue lifting.

[Return to Session 3 check](#)